

GEACX1 Opration Manual

目录

简介

Introduction

接口说明

Interface Description

正面接口

Front interface

配套线束：（选配）

Matching harness: (optional)

左侧接口

Left side interface

配套线束：（选配）

Matching harness: (optional)

右侧接口

Right side interface

背面接口

Rear interface

配套线束：（选配）

Matching harness: (optional)

系统介绍

System introduction

登录信息

Login information

默认安装库

Default installation library

版本查询

nvidia 计算库版本查询

Version query

nvidia compute library version query

kernel 版本信息查询

Querying the kernel version information

DTB 版本信息查询

Querying DTB version information

系统烧录

System burning

使用场景

Usage scenario

烧录接口实物图

Burning interface physical diagram

操作步骤

Operation procedure

准备事项

进入烧录状态

解压固件

烧录
进行烧录
burning
验证烧录结果
开关机
Switch on and off
功率模式设定
Power mode setting

功能介绍

Function introduction

接口特性
Interface characteristic
相机线束 (Camera harness)
名词解释
Noun interpretation
GMSL/GMSL2 相机支持型号
GMSL/GMSL2 camera Supported models
非网页配置方法
Non-web configuration method
相机触发配置工具使用
Use the camera trigger configuration tool
内部触发GMSL相机工具
Internal trigger GMSL camera tool
显示相机帧率 (Displays camera frame rate)
保存图像
Save the image
故障排查
troubleshooting

Q1: 相机不出图, 用 show_cuda 看是绿屏或者黄屏的情况
Q2: 相机出图, 图像颜色空间不对

CPLD

CPLD

CPLD 主要功能
Main functions of CPLD
特别注明
specific description
CPLD 固件版本查询及升级
Querying and upgrading the CPLD firmware version
CPLD 功能详细描述

This section describes the CPLD functions

串口通信
Serial communication
同步授时
Synchronous timing
同步触发
Synchronous trigger

技术指标
Technical index
触发信号

触发时间戳

Trigger timestamp

CPU 授时

CPU timing

外部第三方设备授时

External third-party devices provide time

具体使用

Specific use

GPS 授时

GPS timing

外部惯导第三方授时

External inertial navigation third party timing

外部惯导 GPRMC 选择RS232

External inertial navigation GPRMC selects RS232

GPS

简介

Introduction

接线 (Wiring connection)

外接信号采集设备 (GPS 天线)

External signal acquisition equipment (GPS antenna)

GPS 的配置 (GPS configuration)

GPS使用PGPS usage) 使用

安装时间同步服务器chrony (Install the time synchronization server chrony)

安装 gpsd工具集

Install the gpsd toolset

查看 GPS 输出

View GPS output

验证授时是否成功 (Verify that the timing is successful)

故障排查 (troubleshooting)

CAN

CAN

CAN配置方法 (CAN configuration method)

设置波特率

Set baud rate

can发送数据

can send data

canfd 配置方法 (canfd configuration method)

canfd 发送数据 (canfd sends data)

canfd 接收数据 (canfd accepts data)

can 使用 (can usr)

故障排查 (troubleshooting)

UART

串口种类及区别

Serial port types and differences

PIN串口使用 (PIN serial port usage)

引脚定义

Pin definition

使用方法

步骤一：机器端配置 (minicom 工具)

Usage method

Step 1: Machine side configuration (minicom tool)

步骤二：电脑端配置

Step 2: Configure the PC

步骤三：收发检验

Step 3: Send and receive inspection

3. Use the serial ports simultaneously

网络使用

Internet use

网口信息资源

Network port information resource

网络线束

Network harness

万兆以太网线束

10 Gigabit Ethernet harness

线束实物图

Diagram of wire harness

线束规格

Harness specification

千兆以太网线束

线束实物图

Gigabit Ethernet harness

Diagram of wire harness

线束规格

网络测试

Network test

配置静态IP（使用 netplan）

Configuring a static IP Address (using netplan)

操作步骤

修改配置文件

Modify the configuration file

验证 IP(Authentication IP)

故障排查(troubleshooting)

4G

天线图（Antenna diagram）

模块确认（Module confirmation）

使用步骤（Procedure of use）

1. 下载并安装最新版的4G服务包

1. Download and install the latest 4G service pack

2. 安装4G天线

2. Install the 4G antenna

3. 插入SIM卡

3. Insert the SIM card

4. 断电重启

4. Power off and restart

验证服务（Verification service）

查看4G服务运行状态（View the 4G service running status）

查询4G网络对应网口（Query the network port corresponding to the 4G network）

故障排查（troubleshooting）

WIFI

使用步骤

Procedure of use

WIFI天线实物图 (WIFI antenna physical picture)

使用方法

信号查看 (Signal viewing)

sta模式设定

sta mode setting

故障排查 (Troubleshooting)

FAN

风扇控制及使用方法

Fan control and usage methods

MCU

网卡半双工协议适配 (Nic half-duplex protocol adaptation)

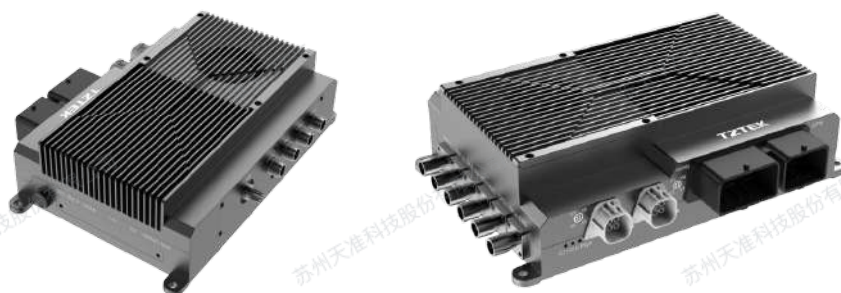
EMMC使用寿命分析 (EMMC Service Life Analysis)

简介

GEACX1是基于NVIDIA Jetson Orin嵌入式GPU模组开发的AI边缘计算设备，具备200TOPS算力，可满足工业应用场景高复杂性、高可靠性的需求。内置的GPS 模块和 CPLD 芯片组成时间同步系统，向控制器的不同处理器和连接到控制器的各个传感器提供UTC时间信息和与 UTC 时间对齐的触发信号，使整个传感、计算和控制系统保持时间同步。

Introduction

GEACX1 is an AI edge computing device developed based on NVIDIA Jetson Orin embedded GPU module, with 200TOPS computing power, which can meet the requirements of high complexity and high reliability in industrial application scenarios. The built-in GPS module and CPLD chip form a time synchronization system, which provides UTC time information and trigger signals aligned with UTC time to different processors of the controller and various sensors connected to the controller, so that the entire sensing, computing and control system remains time synchronization.



接口说明

Interface Description

正面接口

Front interface



从左依次往右，从上往下：

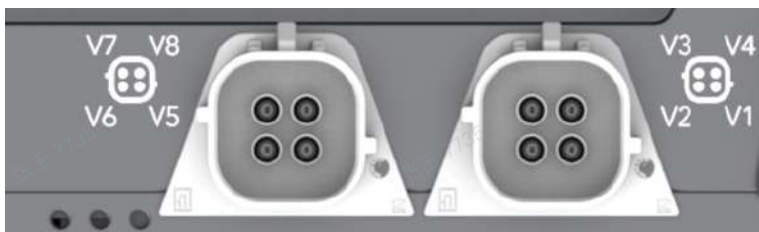
接口名称（丝印）	接口说明	接口信号定义及功能
Serial	串口信号接口	RS232*1\RS485*1\RS422*1\DEBUG*1信号
GPIO	GPIO信号接口	4路GPIO信号
4G-1	4G天线1（TNC-K）接口	接（TNC-J）接口的4G天线（主）
4G-2	4G天线2（TNC-K）接口	接（TNC-J）接口的4G天线（从）
LAN-1	RJ45 千兆以太网接口1	兼容百兆千兆以太网通信（与LAN-2共享千兆带宽）
LAN-2	RJ45 千兆以太网接口2	兼容百兆千兆以太网通信（与LAN-1共享千兆带宽）

From left to right, from top to bottom:

Interface name (Silkscreen)	Interface description	Interface signal definition and function
Serial	Serial signal interface	RS232*1\RS485*1\RS422*1\DEBUG*1 signal

GPIO	GPIO signal interface	Four GPIO signals
4G-1	4G Antenna 1 (TNC-K) port	4G antenna with (TNC-J) interface (main)
4G-2	4G Antenna 2 (TNC-K) port	4G antenna connected to (TNC-J) interface (slave)
LAN-1	RJ45 Gigabit Ethernet port 1	Compatible with Gigabit Ethernet communication (sharing gigabit bandwidth with LAN-2)
LAN-2	RJ45 Gigabit Ethernet port 2	Compatible with Gigabit Ethernet communication (sharing gigabit bandwidth with LAN-1)

接口详解:



接口名称 (丝印)	接口说明	接口信号定义及功能	接口说明
V5~8	1	S_INA+_3	从GMSL #V5
	2	S_INB+_3	从GMSL #V6
	3	S_INA+_4	从GMSL #V7
	4	S_INB+_4	从GMSL #V8
V1~4	1	S_INA+_1	从GMSL #V1
	2	S_INB+_1	从GMSL #V2
	3	S_INA+_2	从GMSL #V3
	4	S_INB+_2	从GMSL #V4

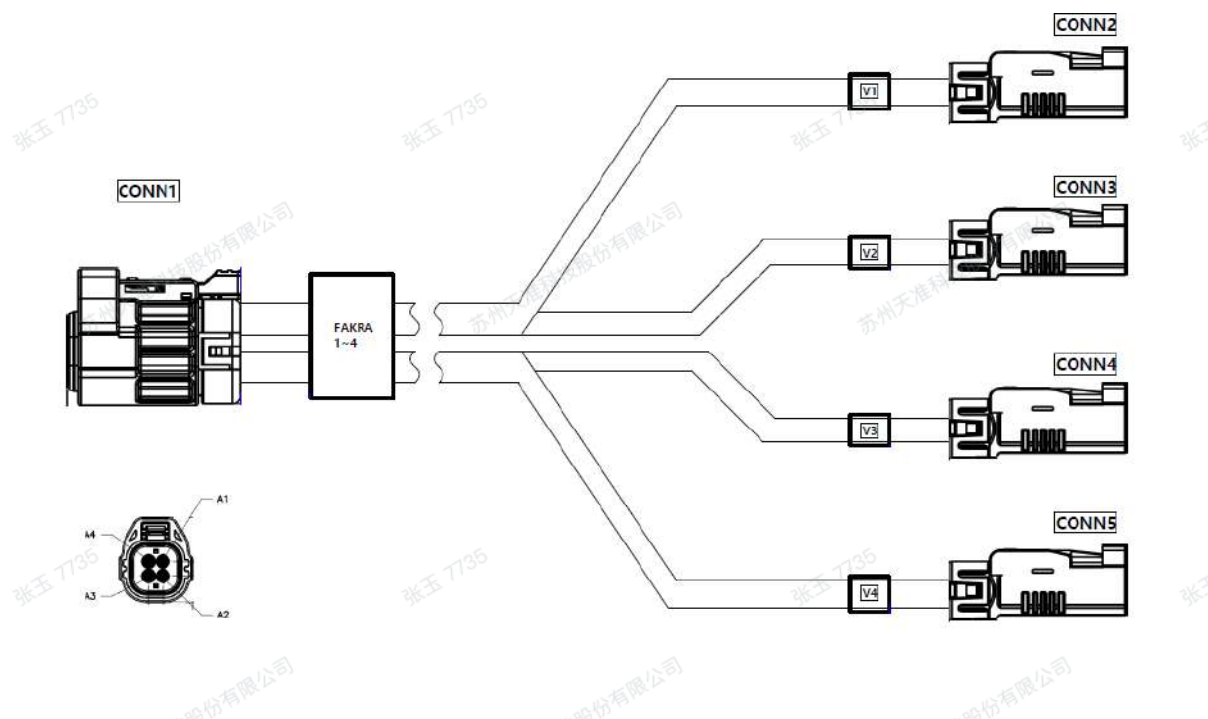
Interface detail:

Interface name (Silkscreen)	Interface serial number	Interface signal definition and function	Interface description
V5~8	1	S_INA+_3	从GMSL #V5
	2	S_INB+_3	从GMSL #V6
	3	S_INA+_4	从GMSL #V7
	4	S_INB+_4	从GMSL #V8
V1~4	1	S_INA+_1	从GMSL #V1
	2	S_INB+_1	从GMSL #V2
	3	S_INA+_2	从GMSL #V3
	4	S_INB+_2	从GMSL #V4

配套线束：（选配）

Matching harness: (optional)

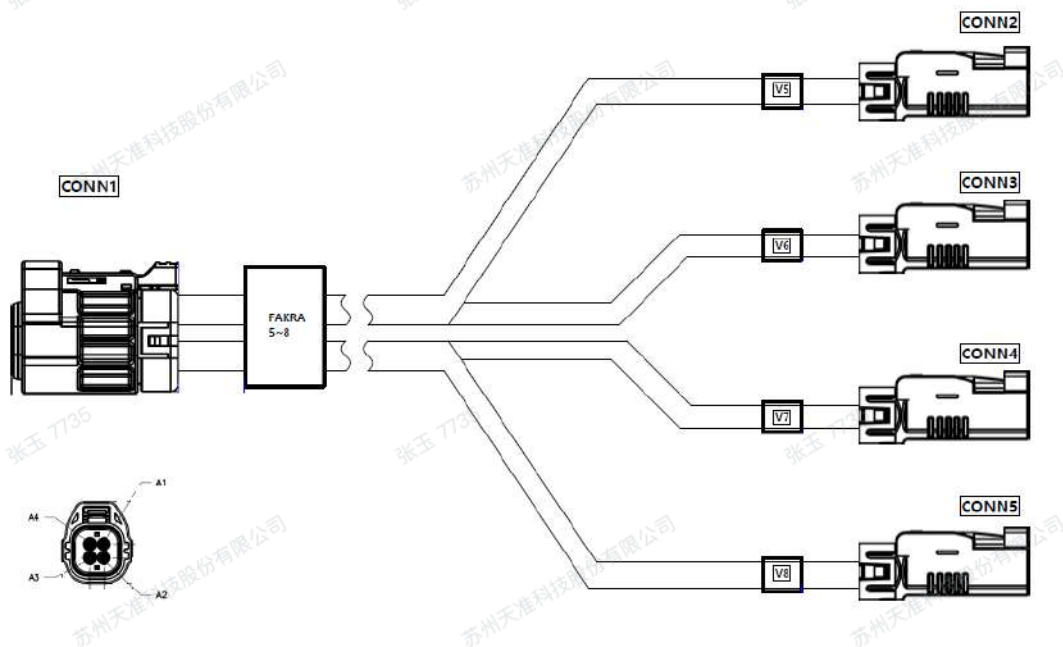
V1-4接口配套线束（Cable harness for the V1-4 port）





连接顺序		
CONN1		标签
A3	CONN2	V1
A2	CONN3	V2
A1	CONN4	V3
A4	CONN5	V4

V5-8接口配套线束 (Cable harness for V5-8 interface)



连接顺序		
CONN1		标签
A3	CONN2	V5
A2	CONN3	V6
A1	CONN4	V7
A4	CONN5	V8

V1-4/V5-8对应线束实物图 (V1-4/V5-8 corresponds to the physical diagram of the wire harness)



四合1接口线束实物图

品牌：电连

型号：818024869 (板端)818018737 (线端)

Four-in-one 1 interface wiring harness physical diagram

Brand: Electric Connection

Model: 818024869 (plate end)818018737 (wire end)



接口名称	引脚序号	接口信号定义	接口说明
48PIN	A1	RS232_TX_GPRMC_CON_1	CPLD串口输出GPRMC信号，232电平，波特率9600bps(备用RS232_TX_GPRMC口)
	B1	RS232_RX_GPRMC_CON_1	不支持

C1	RS232_TX_GPRMC_CON	CPLD串口输出GPRMC信号，232电平，波特率9600bps
D1	RS232_RX_GPRMC_CON	可通过配置串口，配置CPLD接受外部惯导输入的GPS的GPRMC信号，232电平，波特率115200bps
E1	RS485_A_CON_1_R	默认CPLD串口输出GPS的GPRMC信号，485电平，波特率9600bps
F1	RS485_B_CON_1_R	默认CPLD串口输出GPS的GPRMC信号，485电平，波特率9600bps
J1	GPIO_BIDIR6_C	做为GPI，GSML相机使用外部触发信号输入
K1	GPIO_BIDIR5_C	做为GPI，GSML相机使用外部触发信号输入
A2	GND	地
B2	LIN4_RX	TC397 MCU 发送
C2	LIN4_TX	TC397 MCU 接收
D2	GND	地
E2	MCU_OUT_4_12V	MCU输出
F2	MCU_OUT_3_12V	MCU输出
G2	MCU_OUT_2_12V	MCU输出
H2	MCU_OUT_1_12V	MCU输出
J2	PPS_IN2	可通过配置串口，使CPLD接受外部惯导输入的PPS信号，TTL电平，默认接收惯导PPS输入
K2	CPLD_PPS_L_OUT_CON	PPS OUT
A3	GPIO_BIDIR4_C	做为GPI，GSML相机使用外部触发信号输入
B3	GPIO_BIDIR3_C	做为GPI，GSML相机使用外部触发信号输入

C3	GPIO_BIDIR2_C	做为GPI，GSML相机使用外部触发信号输入
D3	GPIO_BIDIR1_C	做为GPI，GSML相机使用外部触发信号输入
E3	GPIO_OUTPUT2_OUT	ORIN引出来的GPO
F3	GPIO_OUTPUT1_OUT	ORIN引出来的GPO
G3	GPIO_INPUT2_C	ORIN引出来的GPI
H3	GPIO_INPUT1_C	ORIN引出来的GPI
J3	RS485_2_A1_CON_2_RS232_1_TX_R	出厂默认RS232接口 (通过重工主板的方式，可以改为RS485)
K3	RS485_2_B1_CON_2_RS232_1_RX_R	出厂默认RS232接口 (通过重工主板的方式，可以改为RS485)
A4	RS232_TX_DEBUG_CON	RS232_TX_DEBUG口
B4	RS232_RX_DEBUG_CON	RS232_RX_DEBUG口
C4	RS485_1_A1_CON_1_RS232_2_TX_R	出厂默认RS485接口 (通过重工主板的方式，可以改为RS232)
D4	RS485_1_B1_CON_1_RS232_2_RX_R	出厂默认RS485接口 (通过重工主板的方式，可以改为RS232)
E4	M_CAN1_H_CON	M_CAN1_H (核心板原生CAN1)
F4	M_CAN1_L_CON	M_CAN1_L(核心板原生CAN1)
G4	M_CAN0_H_CON	M_CAN0_H(核心板原生CAN0)
H4	M_CAN0_L_CON	M_CAN0_L(核心板原生CAN0)
J4	SPI_CANL_CON(功能取消)	SPI_CANL (功能取消)
K4	SPI_CANH_CON(功能取消)	SPI_CANH (功能取消)
L1	GND	地

L2	GND	地
L3	GND	地
L4	GND	地
M1	/	
M2	/	
M3	/	
M4	/	

Interface name	Pin number	Interface signal definition	Interface description
48PIN	A1	RS232_TX_GPRMC_CON_1	CPLD serial port output GPRMC signal, 232 level, baud rate 9600bps(standby RS232_TX_GPRMC port)
	B1	RS232_RX_GPRMC_CON_1	nonsupport
	C1	RS232_TX_GPRMC_CON	CPLD serial port output GPRMC signal, 232 level, baud rate 9600bps
	D1	RS232_RX_GPRMC_CON	By configuring the serial port, the CPLD can be configured to accept the external inertial navigation input GPS GPRMC signal, 232 level, baud rate 115200bps
	E1	RS485_A_CON_1_R	The default CPLD serial port output GPS GPRMC signal, 485 level, baud rate 9600bps
	F1	RS485_B_CON_1_R	The default CPLD serial port output GPS GPRMC signal, 485 level, baud rate 9600bps
	J1	GPIO_BIDIR6_C	As a GPI, the GSML camera uses an external trigger signal input

K1	GPIO_BIDIR5_C	As a GPI, the GSML camera uses an external trigger signal input
A2	GND	ground
B2	LIN4_RX	TC397 MCU send
C2	LIN4_TX	TC397 MCU receive
D2	GND	group
E2	MCU_OUT_4_12V	MCU input
F2	MCU_OUT_3_12V	MCU output
G2	MCU_OUT_2_12V	MCU output
H2	MCU_OUT_1_12V	MCU output
J2	PPS_IN2	You can configure the serial port to enable the CPLD to accept the external inertial navigation device (INS) input PPS signal, TTL level, and receive the INS PPS input by default
K2	CPLD_PPS_L_OUT_CON	PPS OUT
A3	GPIO_BIDIR4_C	As a GPI, the GSML camera uses an external trigger signal input
B3	GPIO_BIDIR3_C	As a GPI, the GSML camera uses an external trigger signal input
C3	GPIO_BIDIR2_C	As a GPI, the GSML camera uses an external trigger signal input
D3	GPIO_BIDIR1_C	As a GPI, the GSML camera uses an external trigger signal input
E3	GPIO_OUTPUT2_OUT	The GPO that ORIN brought out

F3	GPIO_OUTPUT1_OUT	The GPO that ORIN brought out
G3	GPIO_INPUT2_C	The GPI that ORIN brought out
H3	GPIO_INPUT1_C	The GPI that ORIN brought out
J3	RS485_2_A1_CON_2_RS232_1_TX_R	Factory default RS232 interface (Through the heavy motherboard, you can change to RS485)
K3	RS485_2_B1_CON_2_RS232_1_RX_R	Factory default RS232 interface (Through the heavy motherboard, you can change to RS485)
A4	RS232_TX_DEBUG_CON	RS232_TX_DEBU PORT
B4	RS232_RX_DEBUG_CON	RS232_RX_DEBUG PORT
C4	RS485_1_A1_CON_1_RS232_2_TX_R	The factory default RS485 port (By way of heavy motherboard, it can be changed to RS232)
D4	RS485_1_B1_CON_1_RS232_2_RX_R	The factory default RS485 port (By way of heavy motherboard, it can be changed to RS232)
E4	M_CAN1_H_CON	M_CAN1_H (Core plate native CAN1)
F4	M_CAN1_L_CON	M_CAN1_L(Core plate native CAN1)
G4	M_CAN0_H_CON	M_CAN0_H(Core plate native CAN0)
H4	M_CAN0_L_CON	M_CAN0_L(Core plate native CAN0)
J4	SPI_CANL_CON(Function cancellation)	SPI_CANL (Function cancellation)
K4	SPI_CANH_CON(Function cancellation)	SPI_CANH (Function cancellation)
L1	GND	group
L2	GND	group
L3	GND	group

L4	GND	group
M1	/	
M2	/	
M3	/	
M4	/	



接口名称	引脚序号	接口信号定义	接口说明
32PIN	A1	CAN10_H_CON	CAN10_H
	B1	CAN10_L_CON	CAN10_L
	C1	CAN3_H_CON	CAN3_H
	D1	CAN3_L_CON	CAN3_L
	E1	CAN11_H_CON	CAN11_H
	F1	CAN11_L_CON	CAN11_L
	A2	CAN1_H_CON	CAN1_H
	B2	CAN1_L_CON	CAN1_L
	C2	CAN0_H_CON	CAN0_H
	D2	CAN0_L_CON	CAN0_L
	E2	CAN2_H_CON	CAN2_H
	F2	CAN2_L_CON	CAN2_L

A3	MCU_IN1	MCU_IN输入 (MCU_IN INPUT)
B3	MCU_IN2	MCU_IN输入 (MCU_IN INPUT)
C3	MCU_IN3	MCU_IN输入 (MCU_IN INPUT)
D3	MCU_IN4	MCU_IN输入 (MCU_IN INPUT)
E3	IGN	开关点火信号(Switch ignition signal)
F3	GND	地(group)
A4	CAN13_H_CON	CAN13_H
B4	CAN13_L_CON	CAN13_L
C4	MCU_CAN_H_CON	MCU_CAN_H
D4	MCU_CAN_L_CON	MCU_CAN_L
E4	LIN1	LIN口 (LIN port)
F4	LIN2	LIN口 (LIN port)
H1	POWER	电源 (Power source)
H2	POWER	电源 (Power source)
H3	POWER	电源 (Power source)
H4	/	
G1	GND	地 (group)
G2	GND	地 (group)
G3	GND	地 (group)
G4	GND	地 (group)



电源线 接线表				
⑦	A	B	线上标签	线色
H1	P1		POWER1	红
H2	P2			红
G1	P3			黑/黄绿
G2	P4			黑/黄绿
H3		P1	POWER2	红
G3		P3		黑/黄绿
G4		P4		黑/黄绿

设备上电

- 1: 将32PIN线束接到设备上
- 2: 使用9~36V电源连接电源线
- 3: 给USB接口供电（供5V直流电）
- 4: 将3档开关拨至O档
- 5: 完成设备上电

Power on the device

- 1: Connect the 32PIN harness to the device
- 2: Connect the power cable using a 9-36 V power supply

- 3: Power the USB interface (5V DC)
- 4: Switch the 3rd gear to the O gear
- 5: Power on the device

点火开关 (Ignition switch)

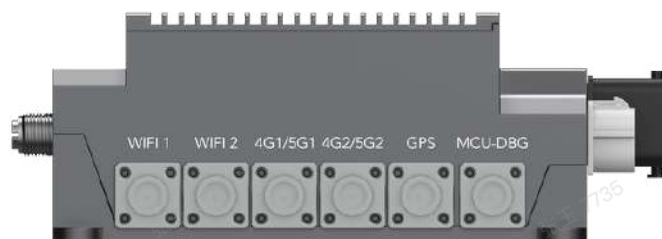


将开关置挡位 **O** ,设备即可上电

Set the switch to gear **O** , and the device can be powered on

左侧接口

从左依次往右, 从上往下:



接口	接口型号	接口说明
WIFI1	TNC-K接口	WIFI1天线接口
WIFI2	TNC-K接口	WIFI2天线接口
5G1/4G1	TNC-K接口	5G1/4G1天线接口
5G2/4G2	TNC-K接口	5G2/4G2天线接口

GPS	TNC-K接口	GPS天线接口
MCU-DBG	M8-8pin (公针) 接口	MCU调试烧录接口

Left side interface

From left to right, from top to bottom:

port	Interface type	Interface description
WIFI1	TNC-K interface	WIFI1 antenna port
WIFI2	TNC-K interface	WIFI2 antenna port
5G1/4G1	TNC-K interface	5G1/4G1 antenna port
5G2/4G2	TNC-K interface	5G2/4G2 antenna port
GPS	TNC-K interface	GPS antenna port
MCU-DBG	M8-8 PIN (male pin) interface	MCU debug burning interface

配套线束：（选配）

Matching harness: (optional)

TCN-J接口 天线（胶棒、磁吸、贴片等可选）

TCN-J interface antenna (glue stick, magnetic suction, patch, etc. Optional)



MCU-DBG（MCU调试烧录接口）

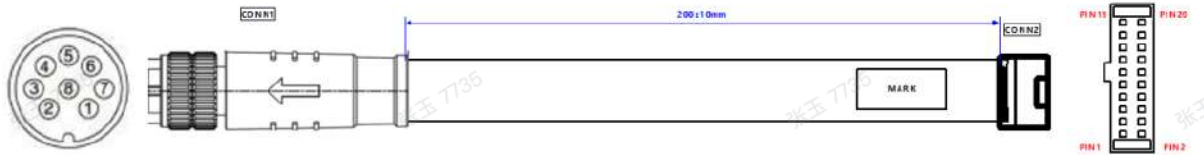
MCU-dbg (MCU debug and burn interface)



PIN位 (PIN position)	定义 (definition)
1	TCK/DAP0
2	GND
3	TDO/DAP2
4	TDI/DAP3
5	3.3V
6	TMS/DAP1
7	PORST#
8	T RST

配套线束 (可选)

Matching harness (optional)



CONN1	CABLE(颜色)	定义	CONN2
1	白	TCK/DAP0	9
2	棕	GND	4
3	绿	TDO/DAP2	13
4	黄	TDI/DAP3	5
5	灰	3.3V	1
6	粉	TMS/DAP1	7
7	蓝	PORST	3
8	红	T RST	15

右侧接口

从左依次往右，从上往下：



接口	接口型号	接口说明
RST	RST按钮	机器重置键
RECV	RECV按钮	机器恢复键（用于烧录）
HDMI	视频输出	核心板HDMI 2.0
U3	USB3.0接口	USB3.0接口
U2	USB2.0接口	USB2.0-OTG接口，可作为烧录口
NANO-SIM	NANO-SIM卡槽	NANO-SIM卡槽

Right side interface

From left to right, from top to bottom:

port	Interface type	Interface description
RST	RSTbottom	Machine reset key
RECV	RECVbottom	Machine recovery key (for burning)
HDMI	Video output	Core board HDMI 2.0
U3	USB3.0 interface	USB3.0 interface
U2	USB2.0 interface	USB2.0-OTG interface, which can be used as a burning port
NANO-SIM	NANO-SIM card slot	NANO-SIM card slot

背面接口

从左依次往右，从上往下：



接口	接口型号	接口说明
10G	M12-X CODE接口（母孔）	万兆网络接口
LAN1	M12-A CODE接口（公针）	千兆网络接口
LAN2	M12-A CODE接口（公针）	千兆网络接口
LAN3	M12-A CODE接口（公针）	千兆网络接口
LAN4	M12-A CODE接口（公针）	千兆网络接口

Rear interface

From left to right, from top to bottom:

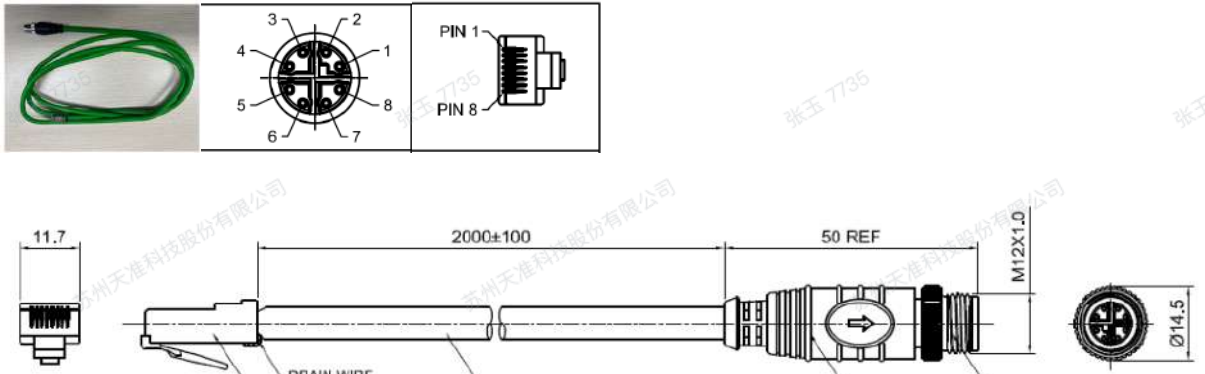
port	Interface type	Interface description
10G	M12-X CODE Interface (female)	Ten Gigabit network interface
LAN1	M12-A CODE Interface (Male Pin)	Gigabit network interface
LAN2	M12-A CODE Interface (Male Pin)	Gigabit network interface
LAN3	M12-A CODE Interface (Male Pin)	Gigabit network interface
LAN4	M12-A CODE Interface (Male Pin)	Gigabit network interface

配套线束：（选配）

10G 配套线束

Matching harness: (optional)

10G matching harness



M12-CAT6-RJ45		
M12 CONNECTOR	CAT6 CABLE	RJ45 CONNECTOR
NUMBER	COLOR	NUMBER
1	WHITE/ORANGE STRIPE	1
2	ORANGE	2
3	WHITE/GREEN STRIPE	3
4	GREEN	6
5	WHITE/BROWN STRIPE	7
6	BROWN	8
7	WHITE/BLUE STRIPE	5
8	BLUE	4

LAN配套线束（选配）

LAN Harness (optional)



M12-A端线序	1	2	3	4	5	6	7	8
----------	---	---	---	---	---	---	---	---

RJ45端线序	5	7	8	2	3	1	4	6
线色	白蓝	白棕	棕	橙	白绿	白橙	蓝	绿

M12-A Cable sequence	1	2	3	4	5	6	7	8
RJ45 end cable sequence	5	7	8	2	3	1	4	6
Line color	White and blue	White and brown	brown	orange	White and green	White and orange	blue	green

系统介绍

System introduction

登录信息

天准设备采用Ubuntu 20.04系统。

Login information

Tianjun device uses Ubuntu 20.04 system.

默认安装库

软件名称	版本信息
Jetpack	5.0.2
cuDNN	8.4.1.50
TensorRT	8.4.1.5
Opencv(with cuda)	4.5.4
VPI	2.1.6
Vulkan	1.3.203

Default installation library

Software name	Version information
Jetpack	5.0.2
cuDNN	8.4.1.50
TensorRT	8.4.1.5
Opencv(with cuda)	4.5.4
VPI	2.1.6
Vulkan	1.3.203

版本查询

nvidia 计算库版本查询

```
jetson_release

#结果(示例,以实际信息为准)

* NV Power Mode: MAXN - Type: 0

* jetson_stats.service: active

- Libraries:

* cuDNN: 8.3.2.49

* TensorRT: 8.4.0.9

* OpenCV: 4.5.4 compiled CUDA: NO

* VPI: 2.0.9

* Vulkan: 1.3.203
```

Version query

nvidia compute library version query

```
jetson_release

#结果(示例,以实际信息为准)
```

```
* NV Power Mode: MAXN - Type: 0

* jetson_stats.service: active

- Libraries:

* cuDNN: 8.3.2.49

* TensorRT: 8.4.0.9

* OpenCV: 4.5.4 compiled CUDA: NO

* VPI: 2.0.9

* Vulkan: 1.3.203
```

kernel 版本信息查询

```
cat /proc/version
```

#结果(示例,以实际信息为准)

```
Linux version 5.10.104-tegra (fengyi@fengyi) #189 SMP PREEMPT Thu Sep 8 10:48:05 CST 2022 tztek_version: [product:geac91, hardware:510jx0_r2_0,jetpack:jp5.0.2_ga, soft:v1.0, type:release]
```

Querying the kernel version information

```
cat /proc/version
```

Example: Actual

```
information prevails. Linux version 5.10.104-tegra (fengyi@fengyi) #189 SMP PREEMPT Thu Sep 8 10:48:05 CST 2022 tztek_version: [product:geac91, hardware: 510jx0_r2_0,jetpack: jp5.0.2_ga, soft:v1.0, type:release]
```

DTB 版本信息查询

查询结果（示例，以实际信息为准）

```
dmesg | grep DTB
```

#查询结果（示例，以实际信息为准）

```
[ 0.461339] DTB Build time:Sep 8 2022 20:59:08

[ 0.462454] DTB Build version: 510jx0_r2_0_jp5.0.2_ga_v1.0_release
```

Querying DTB version information

Query result (Example, based on the actual information)

```
dmesg | grep DTB  
  
# Example: Actual information prevails.  
  
[ 0.461339] DTB Build time:Sep 8 2022 20:59:08  
  
[ 0.462454] DTB Build version: 510jx0_r2_0_jp5.0.2_ga_v1.0_release
```

系统烧录

System burning

使用场景

需要更换系统。

Usage scenario

Need to replace system

烧录接口实物图

Burning interface physical diagram



GEACX1 有 U2 一个系统烧录口。

GEACX1 has a system burner for U2.

操作步骤

Operation procedure

准备事项

1. 装有 Ubuntu 系统的 x86 机器一台

2. USB2.0 转 micro-usb 数据线一根

3. GEACX1 机器一台

4. 系统固件包（可咨询销售或者FAE）

preparation

1. An x86 machine with Ubuntu installed

2. USB2.0 to micro-usb data cable

3. One GEACX1 machine

4. System firmware package (consult sales or FAE)

进入烧录状态

方法一：

1. micro-usb 连接 GEACX1 和 x86 电脑

2. GEACX1 开机进入系统

3. 打开终端，输入指令

```
sudo reboot --force forced-recovery
```

Enter the burn state

Method 1:

1. The micro-usb connects the GEACX1 and x86 computers

2. Start GEACX1 and log in to the system

3. Open the terminal and enter commands

[复制代码](#)

```
sudo reboot --force forced-recovery
```

方法二:

1. micro-usb 连接 GEACX1和 x86 电脑
2. 机器通电
3. 按住 recovery 按键，不松开
4. 按下 reset 按键1~3秒后松开

在 x86 主机上输入 lsusb ，如果出现 NVIDIA 设备，则成功进入烧录模式。

```
wunin@wunin-Latitude-5411:~$ lsusb
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 008: ID 0955:7019 NVIDIA Corp. APX
Bus 001 Device 004: ID 8087:0026 Intel Corp.
Bus 001 Device 003: ID 0c45:0720 Microdia Integrated Webcam HD
Bus 001 Device 002: ID 413c:301a Dell Computer Corp. Dell M5116 USB Optical Mouse
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
wunin@wunin-Latitude-5411:~$
```

注：每次烧录都要重新进入烧录状态。

Method 2:

1. The micro-usb connects the GEACX1 and x86 computers
2. Power on the machine
3. Press and hold the recovery button
4. Press the reset button for 1 to 3 seconds and release it

Enter lsusb on an x86 host and enter burn mode successfully if an NVIDIA device appears.

Note: Re-enter the burn state for each burn.

解压固件

步骤一：准备好系统包：xxxx.tar.gz（**注意：**一定要找对要烧录的版本，找对安装包）

步骤二：解压系统包

```
#打开终端输入代码

sudo apt-get install python --首次安装输入命令

sudo apt-get install libxml2-utils --安装依赖环境（若环境已存在则此步骤可省略）

#解压

sudo tar -xpf XXXX.tar.gz（安装包名）
```

步骤三：进入解压后的固件目录

烧录

```
cd flashtool_tztek_geac_orin_jp5.0.2_ga_510jx0_v1.0/
```

步骤四：继续进入到解压文件的 Linux_for_Tegra 文件中：

```
#输入

cd Linux_for_Tegra/

ls
```

效果如下：


```

zhouhailong@zhouhailong-Latitude-5421:~/flashtool_jp5.0.0_GA_jx0_r2.0_v1.0/Linux_for_Tegra$ ls
apply_binaries.sh      holoscan-devkit.conf      p2822-0000+p2888-0001.conf
burn_image.sh          jetson-agx-orin-devkit-as-jao-32gb.conf  p2822-0000+p2888-0004.conf
build_l4t_bup.sh       jetson-agx-orin-devkit-as-nx-16gb.conf    p2822-0000+p2888-0008.conf
clara-agx-xavier-devkit.conf  jetson-agx-orin-devkit-maxn.conf          p2822-0000+p2888-0008-maxn.conf
e3900-0000+p2888-0004-b00.conf jetson-agx-xavier-devkit.conf              p2822-0000+p2888-0008-noecc.conf
flash.sh               jetson-agx-xavier-ind-noecc.conf           p2972-0000.conf.common
flash_all.sh           jetson-agx-xavier-industrial-mxn.conf      p2972-0000-devkit-maxn.conf
holoscan-devkit.conf   jetson-xavier.conf                    p2972-0000-devkit-slvs-ec.conf
jetson-agx-orin-devkit-as-jao-32gb.conf  jetson-xavier-maxn.conf                  p2972.conf
jetson-agx-orin-devkit-as-nx-16gb.conf    jetson-xavier-nx-devkit.conf             p3449-0000+p3668-0000-qspt-sd.conf
jetson-agx-orin-devkit-maxn.conf          jetson-xavier-nx-devkit-emmc.conf         p3449-0000+p3668-0001-qspt-emmc.conf
jetson-agx-xavier-devkit.conf             jetson-xavier-nx-devkit-qspt.conf         p3509-0000+p3668-0000-qspt.conf
jetson-agx-xavier-ind-noecc.conf          jetson-xavier-slvs-ec.conf               p3509-0000+p3668-0000-qspt-sd.conf
jetson-agx-xavier-industrial-mxn.conf     jetson-xavier-slvs-ec.conf               p3509-0000+p3668-0001-qspt-emmc.conf
jetson-xavier.conf                       README_AutoFlash.txt                    p3668.conf.common
jetson-xavier-maxn.conf                   source_sync.sh                          p3701.conf.common
jetson-xavier-nx-devkit.conf               tztek_flash.sh                         p3711-0000+p3701-0002.conf
jetson-xavier-nx-devkit-emmc.conf          tztek_flash.sh                         p3737-0000+p3701-0000-as-p3701-0004.conf
jetson-xavier-nx-devkit-qspt.conf          tztek_flash.sh                         p3737-0000+p3701-0000-as-p3767-0000.conf
jetson-xavier-slvs-ec.conf                 tztek_flash.sh                         p3737-0000+p3701-0000-as-p3767-0001.conf
jetson-xavier-slvs-ec.conf                 tztek_flash.sh                         p3737-0000+p3701-0000.conf
l4t_generate_soc_bup.sh                   tztek_flash.sh                         p3737-0000+p3701-0000-maxn.conf
l4t_sign_image.sh                        tztek_flash.sh                         README_AutoFlash.txt
nvautoflash.sh                           tztek_flash.sh                         source_sync.sh
nvsdmanager_flash.sh                     tztek_flash.sh                         source_sync.sh
nvtools.sh                               tztek_flash.sh                         tztek_flash.sh

```

进行烧录

burning

步骤一：执行烧录脚本

Step 1: Execute the burn script

#输入以下代码进行烧录 (#Enter the following code to burn)

```
sudo ./tztek_flash.sh
```

步骤二：选择操作

脚本支持多个功能，包括备份系统、烧录系统、更新内核、更新设备树等。

这里选择 8. flash-all-with-system，进行完整系统烧录（固件中已包含system文件系统）。

Step 2: Select an action

Scripts support multiple functions, including backing up the system, burning the system, updating the kernel, updating the device tree, and so on.

Here, select 8.flash-all-with-system for full system burning (the system file system is already included in the firmware).

步骤三：选择平台

脚本支持多个平台，这里选择 6. orin 平台。

Step 3: Select a platform

The script supports multiple platforms, for example, the 6.orin platform.

```
zhouhailong@zhouhailong-Latitude-5421: /flashtool_jp5.0.0_GA_jx0_r2.0_v1.0/Linux_for_Tegra$ sudo ./tztek_flash.sh
[sudo] zhouhailong 的密码:
=====
Welcome to tztek flash tool
v2.1.2
=====
请输入烧录功能编号:
1.kernel 2.dtb 3.backup-system 4.bct 5.cbboot 6.tztek_env 7.Flash-all 8.Flash-all-with-system 9.App_b
number:8
===== flash-all-with-system

请输入平台型号编号:
1.xavier 2.nx-emmc 3.nx-sdcard 4.tx2-8g 5.tx2-4g 6.orin
number:6
===== orin;

----- sudo ./flash.sh -r jetson-agx-orin-devkit mmcblk0p1
(yes/no, default is yes):
```

步骤四：开始烧录

Step 4: Start burning

```
[ 18.4222 ] Writing partition spe-fw_b with spe_t194_sigheader.bin.encrypt
[ 18.4434 ] [.....] 100%
[ 18.4468 ] Writing partition mb2 with nvtboot_t194_sigheader.bin.encrypt
[ 18.4678 ] [.....] 100%
[ 18.4734 ] Writing partition mb2_b with nvtboot_t194_sigheader.bin.encrypt
[ 18.4975 ] [.....] 100%
[ 18.5023 ] Writing partition mts-preboot with preboot_c10_prod_cr_sigheader.bi
n.encrypt
[ 18.5254 ] [.....] 100%
[ 18.5268 ] Writing partition mts-preboot_b with preboot_c10_prod_cr_sigheader.
bin.encrypt
[ 18.5460 ] [.....] 100%
[ 18.5476 ] Writing partition SMD with slot_metadata.bin
[ 18.5671 ] [.....] 100%
[ 18.5865 ] Writing partition SMD_b with slot_metadata.bin
[ 18.5998 ] [.....] 100%
[ 18.6013 ] Writing partition VER_b with emmc_bootblob_ver.txt
[ 18.6134 ] [.....] 100%
[ 18.6145 ] Writing partition VER with emmc_bootblob_ver.txt
[ 18.6272 ] [.....] 100%
[ 18.6283 ] Writing partition master_boot_record with mbr_1_3.bin
[ 18.6412 ] [.....] 100%
[ 18.6422 ] Writing partition APP with system.img
[ 18.6494 ] [.....] 002%
```

```
t
[ 879.9880 ] [.....] 100%
[ 880.0060 ]
[ 880.0060 ] Flashing completed
[ 880.0060 ] Coldbooting the device
[ 880.0086 ] tegrarcn_v2 --isnb2
[ 880.4048 ]
[ 880.4079 ] tegradevflash_v2 --reboot coldboot
[ 880.4105 ] Bootloader version 01.00.0000
[ 880.8105 ]
*** The target t186ref has been flashed successfully. ***
Reset the board to boot from internal eMMC.
```

验证烧录结果

烧录成功之后会自动重启进入系统，建议断电再次重启。

进入系统后，通过如下命令查看版本。

Verify burning results

After successful burning will automatically restart into the system, it is recommended to power off and restart again.

After logging in to the system, run the following command to view the version.

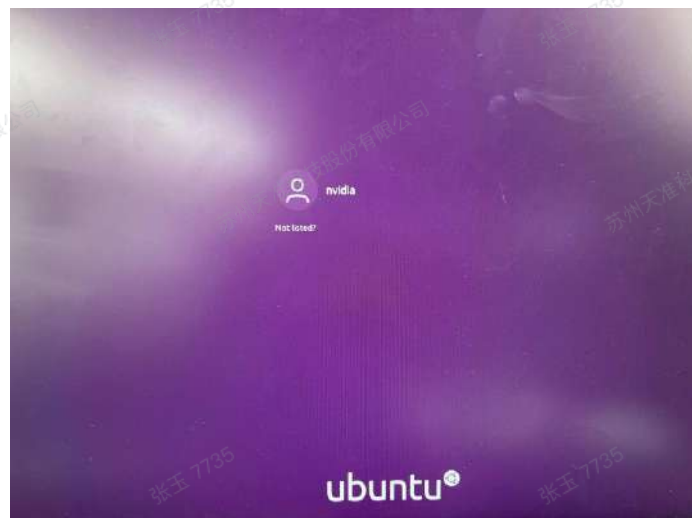
```
cat /proc/dtb  
  
cat /proc/version
```

开关机

开机：天准设备默认开机模式为上电自启动。插入电源，并将显示器通过 HDMI 接口与天准设备相连，开机画面如下图所示。默认登录名为 nvidia，登录密码为 nvidia。

Switch on and off

Power-on: The default power-on mode is automatic startup. Plug in the power supply, and connect the display to the satellite equipment through the HDMI interface, the startup screen is shown below. The default login name and password are nvidia.



关机：在命令行中执行\$ **sudo poweroff**，完成软关机，再切断电源（使用 poweroff 或 shutdown now 关机后机器默认会自动重启，如需要停止请立即断电，此设计为硬件设计须知）。

shutdown: Run \$sudo poweroff in the command line to complete the soft shutdown and then cut off the power. (After poweroff or shutdown now is used, the machine will automatically restart by default. If you need to stop, please poweroff immediately.

重启：在命令行中执行\$ `sudo reboot`，完成重启。

Restart: Run \$`sudo reboot` on the CLI to complete the restart.

功率模式设定

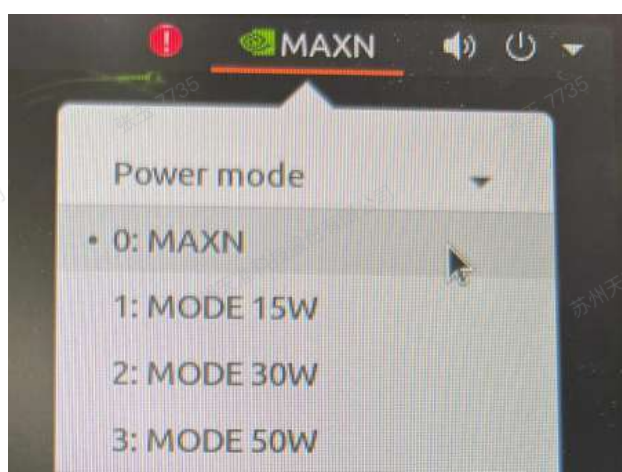
Power mode setting

搭载 Jetson AGX ORIN 的 GEACX1 有多种工作模式。可以通过右上角的 NVIDIA 绿色标志设置进行调整，

GEACX1 默认模式为 0：MAXIN

The GEACX1 with Jetson AGX ORIN operates in a variety of modes. It can be adjusted through the NVIDIA green logo Settings in the upper right corner,

The default mode of GEACX1 is 0: MAXIN



也可采用命令行调整：查看详细信息

You can also use the command line to adjust: View details

#查看设备现在的模式 (View the current mode of the device)

```
sudo nvpmode -q verbose
```

#设定为某一模式 (Set to a pattern)

```
sudo nvpmode -m <MODE ID>
```

#获取在当前模式下的最佳表现 (Get the best performance in the current mode)

sudo jetson_clocks

#查看详细信息 (View details)

sudo jetson_clocks --show

功能介绍

相机接口具体位置在正面接口处，支持 GMSL2/GMSL 相机接入。包含8路 GMSL2 接口，对应接口为 v1~v8，对应软件 video0~video7。

Function introduction

The camera interface is located on the front interface and supports GMSL2/GMSL camera access. Contains eight GMSL2 ports, corresponding to v1 to v8, corresponding to video0 to video7 software.

接口特性

- 不支持热插拔
- 支持15 米同轴电缆的信号传输，如果有更长的线缆需求，请联系商务沟通
- 推荐支持输出分辨率为720p，1080p，4k等多种分辨率的相机

Interface characteristic

- Hot swap is not supported
- Support 15 meters of coaxial cable signal transmission, if there is a longer cable needs, please contact business communication
- It is recommended to support cameras with output resolution of 720p, 1080p, 4k and other resolutions

相机线束 (Camera harness)



相机接入机器时，如图红色框位置方向往上，用力按进去，正确接入时，相机插孔会被卡住。拔出时按住里面的塑料片（红框标注的），切勿暴力拔出。

When the camera is connected to the machine, press the red box upwards. When the camera is connected correctly, the camera jack will be stuck. When pulling out, hold down the plastic sheet inside (marked in the red box), do not pull out violently.



名词解释

- 非触发：摄像头不受外部触发信号控制，自己进行快门；一般相机都支持这种模式。
- 触发：摄像头受外部触发信号控制，进行快门；这种模式需要相机固件支持，购买相机的时候请和厂商确认是否支持。
- fakra Z 接口线束：GMSL/GMSL2 相机接口线束，用于连接相机和 GEACX1 进行数据传输。

Noun interpretation

- Non-trigger: the camera is not controlled by external trigger signals, and the shutter is performed by itself; Most cameras support this mode.
- Trigger: The camera is controlled by an external trigger signal to shutter; This mode requires camera firmware support, please check with the manufacturer when purchasing the camera to see if it is supported.
- fakra Z interface harness: The GMSL/GMSL2 camera interface harness is used to connect the camera to GEACX1 for data transmission

GMSL/GMSL2 相机支持型号

[天准支持相机适配表](#)

首先需要先安装相机相关配置包

下载网址：<http://221.224.80.18:2080/packages/common/debs/>

下载相机配置文件包和相机出图包。

这两个包都有不同的版本，一般建议客户下载最新版本。

GMSL/GMSL2 camera Supported models

tztek supports the camera adaptation table

You need to install the camera-related configuration package first

Download url: <http://221.224.80.18:2080/packages/common/debs/>

Download the camera profile package and camera map package.

Both packages are available in different versions, and customers are generally advised to download the latest version.

1.相机配置文件(Camera profile)

- [tztek-jetson-service-camera-config-510JX0-v2.7.deb](#)

2.相机出图(Camera mapping)

- [tztek-jetson-tool-camera-show-510JX0-v1.0.deb](#)

打开终端，进入到压缩包所在路径，分别执行安装命令(Open the terminal, go to the directory where the compressed package is located, and run the installation command):

```
#安装deb包(Installing the deb package)
sudo dpkg -i tztek-jetson-service-camera-config-510JX0-v1.0.deb --v2.7是版本号
#安装deb包(Installing the deb package)
sudo dpkg -i tztek-jetson-tool-camera-show-510JX0-v1.0.deb
```

非网页配置方法

一般采用非网页配置，本地修改在修改cam_cfg.ini配置之后，再执行初始化相机服务，最后使用下面的命令查看相机初始化结果以达到相机配置目的。

Non-web configuration method

After modifying the cam_cfg.ini configuration, initialize the camera service. At last, use the following command to view the camera initialization result to achieve the camera configuration purpose.

操作 (Controls)	命令 (command)
修改配置 (Modify configuration)	<code>sudo vim /etc/configure-camera/cam_cfg.ini</code>
初始化相机服务 (Initialize the camera service)	<code>sudo systemctl restart jetson-cam-cfg.service</code>
查看初始化log结果 (View the log initialization result)	<code>sudo journalctl -u jetson-cam-cfg.service</code>

若配置示例相机，`sudo vim /etc/configure-camera/cam_cfg.ini` 效果图如下：

To configure a sample camera, `sudo vim /etc/configure-camera/cam_cfg.ini` will look like this:

```
[camera1_2]
hardware_type=xavier
position=v1
serializer=max96717
sensor=imx390
output_pixels=1080p
factory=sensing

[camera3_4]
hardware_type=xavier
position=no_device
serializer=max96705
sensor=ar0231
output_pixels=1080p
factory=sensing

[camera5_6]
hardware_type=xavier
position=no_device
serializer=max96705
sensor=ar0231
output_pixels=1080p
factory=sensing

[camera7_8]
hardware_type=xavier
position=no_device
serializer=max96705
sensor=ar0231
output_pixels=1080p
factory=sensing
-
```

具体配置方法请参考 `/usr/local/bin/configure-camera/cam_cfg.py` 这个里面

For details, see `/usr/local/bin/configure-camera/cam_cfg.py`

相机触发配置工具使用

Use the camera trigger configuration tool

内部触发GMSL相机工具

作用：GEACX1通过 CPU 配置 CPLD 芯片，由 CPLD 产生相机触发信号使能需要触发的相机引脚。

Internal trigger GMSL camera tool

Function: The GEACX1 is configured with the CPLD chip through the CPU, and the CPLD generates the camera trigger signal to enable the camera pin that needs to be triggered

触发包下载（Trigger package download）：

- [tztek-jetson-tool-internal-trigger-camera-v1.0.deb](#)

按顺序输入如下命令（Enter the following commands in sequence）：

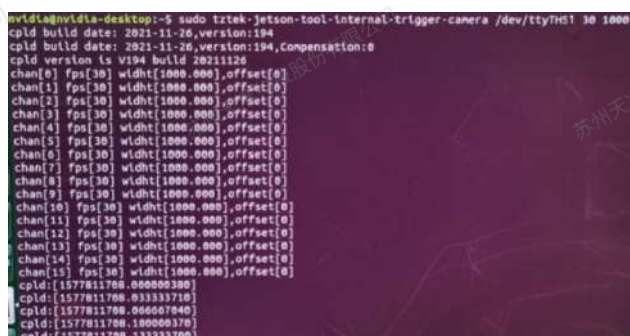
```
#安装deb包（Installing the deb package）

sudo dpkg -i tztek-jetson-tool-internal-trigger-camera-v1.0.deb

#执行触发，其中 ttyTHS1:软件接口，30：30hz，1000：高电平1000μs（Execute trigger, where ttyTHS1: software interface, 30:30
hz, 1000: high level 1000μs）

sudo tztek-jetson-tool-internal-trigger-camera /dev/ttyTHS1 30 1000
```

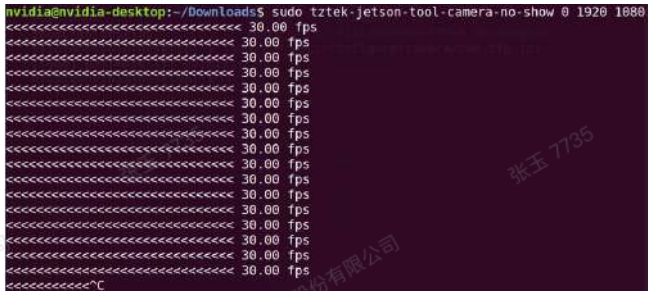
执行成功如下（The execution is successful as follows）：



```
nvdi@nvidia-deaktop:~$ sudo tztek-jetson-tool-internal-trigger-camera /dev/ttyTHS1 30 1000
cpld build date: 2021-11-26,version:194
cpld build date: 2021-11-26,version:194,Compensation:0
cpld version is V194 build 20211126
chan[0] fps[30] width[1000.000],offset[0]
chan[1] fps[30] width[1000.000],offset[0]
chan[2] fps[30] width[1000.000],offset[0]
chan[3] fps[30] width[1000.000],offset[0]
chan[4] fps[30] width[1000.000],offset[0]
chan[5] fps[30] width[1000.000],offset[0]
chan[6] fps[30] width[1000.000],offset[0]
chan[7] fps[30] width[1000.000],offset[0]
chan[8] fps[30] width[1000.000],offset[0]
chan[9] fps[30] width[1000.000],offset[0]
chan[10] fps[30] width[1000.000],offset[0]
chan[11] fps[30] width[1000.000],offset[0]
chan[12] fps[30] width[1000.000],offset[0]
chan[13] fps[30] width[1000.000],offset[0]
chan[14] fps[30] width[1000.000],offset[0]
chan[15] fps[30] width[1000.000],offset[0]
cpld:[1577811708.000000300]
cpld:[1577811708.033333710]
cpld:[1577811708.066667040]
cpld:[1577811708.100000370]
cpld:[1577811708.133333700]
```

注：若出现以下给触发卡在Write ack,reg:00030004,value:255停住，无法继续打印cpld时间戳，可排查下cpld固件版本是否正确安装以及版本是否正确

若版本安装正确可执行sudo tztek-jetson-firmware-cpld-510jx0重新刷写升级cpld固件再尝试执行触发



保存图像

调用测试工具 [tztek-jetson-tool-camera-show-510JX0-v1.0.deb](#) 在本地文件夹保存原始 yuv 图像

操作	命令
保存 v1 摄像头 yuv 图像，v1分辨率 1920*1080	sudo tztek-jetson-tool-camera-no-show-save-images 0 1920 1080
保存 v2 摄像头 yuv 图像，v2分辨率 1920*1080	sudo tztek-jetson-tool-camera-no-show-save-images 1 1920 1080

Save the image

Call the test tool tztek-jetson-tool-camera-show-510JX0-v1.0.deb to save the original YUV image in the local folder

Controls	command
Save v1 camera yuv image, v1 resolution 1920*1080	sudo tztek-jetson-tool-camera-no-show-save-images 0 1920 1080
Save the yuv image of the v2 camera, the v2 resolution is 1920*1080	sudo tztek-jetson-tool-camera-no-show-save-images 1 1920 1080

示例：在使用森云390触发后的相机执行：[sudo tztek-jetson-tool-camera-no-show-save-images 0 1920 1080](#)

Example: Camera execution after firing with Senyun 390:

▼

[复制代码](#)

```
sudo tztek-jetson-tool-camera-no-show-save-images 0 1920 1080
```

执行后在当前路径生成 camera_0_1920_1080.yuv 的原始 yuv 文件图像。（原始图像文件存储较大）

The original yuv file image of camera_0_1920_1080.yuv is generated in the current path. (Original image file storage is large)

故障排查

troubleshooting

Q1：相机不出图，用 show_cuda 看是绿屏或者黄屏的情况

A1：show_cuda 看绿屏或者黄屏，是相机没有数据，两种可能的情况：

情况一：相机初始化失败，相机没有数据，可查看初始化的 log

通过 `sudo journalctl -u jetson-cam-cfg.service` 查看 log。若初始化失败，检查相机型号是否与设置匹配，及实际插的相机位置与写进配置的位置是否匹配。

情况二：相机初始化成功，相机没有数据，可能是此类型相机（一般 max9295 串行 imx390 相机都需要 30hz 触发信号，才可出图）需要触发信号才有数据，可尝试给触发信号参考前面的[相机触发配置工具使用](#)。

若以上操作均未排查不到问题的话，请执行以下几条命令把 log 拍照发给技术支持。

Q1: The camera does not produce a picture. Use the show _ CUDA to see whether the screen is green or yellow.

A1: Look at the green screen or yellow screen show _ CUDA, because the camera has no data. There are two possible situations:

Case 1: Camera initialization fails, and the camera has no data. You can view the initialization log.

View the log via the `sudo journal CTL-u jetson-cam-CFG. Service`. If the initialization fails, check whether the camera model matches the settings, and whether the actual position of the inserted camera matches the position of the written configuration.

Case 2: the camera is initialized successfully, but the camera has no data. It may be that this type of camera (generally, max9295 serial imx390 cameras need a 30 Hz trigger signal to plot) needs a trigger signal to have data. You can try to use the trigger signal by referring to the previous camera trigger configuration tool.

If the above operations fail to find the problem, please execute the following commands to take a photo of the log and send it to the technical support.

```
#查看服务状态 (View service status)

sudo journalctl -u jetson-cam-cfg.service

#打印信息 (Print message)

sudo echo debug > /proc/loglevel && dmesg -w

#cpld版本获取 (Obtain the cpld version)

sudo tztek-jetson-tool-cpld-get-info-510jx0
```

Q2: 相机出图，图像颜色空间不对

A2: show_cuda 看颜色空间不对，是设置图像获取格式与相机输出数据不符，目前我们机器接收相机的数据

UYVY 对应上层设置 YUYV，YUYV 对应上层设置 UYVY，默认接收相机数据为 YUYV。

Q2: The image color space is wrong when the camera is drawing

A2: show_cuda see the color space is wrong, is the set image acquisition format does not match the camera output data, currently our machine receives the camera data

UYVY corresponds to the upper-layer setting YUYV, and YUYV corresponds to the upper-layer setting UYVY. By default, the camera data received is YUYV.

CPLD

CPLD 一般指复杂可编程逻辑器件，主要由逻辑块、可编程互连通道和 I/O 块三部分构成。CPLD 具有编程灵活、集成度高、设计开发周期短、适用范围宽等特点，可实现较大规模的电路设计。GEAC 系统中使用的是 Xilinx 公司生产的 Spartan6 LX16 系列芯片，当前的设计中该芯片的系统工作频率为100Mhz，且具备并行处理的能力。

CPLD

CPLD generally refers to the complex programmable logic device, mainly composed of logic block, programmable interconnect channel and I/O block three parts. CPLD has the characteristics of flexible programming, high integration, short design and development cycle, wide application range, etc. It can realize large-scale circuit design. The Spartan6 LX16 series chips produced by Xilinx company are used in the GEAC system. In the current design, the system frequency of this chip is 100Mhz and it has the capability of parallel processing.

CPLD 主要功能

功能	功能说明	功能补充
1	负责与内部的 CPU 进行串口通信	CPLD 解析 CPU 指令集并完成相应的逻辑控制功能
2	支持多途径对自身授时	- CPU 利用其系统时间对 CPLD 授时 - 内部的 GPS 芯片对 CPLD 授时 - 外部惯导设备对 CPLD 授时
3	CPLD 对外授时	- 给同在GEAC控制器内部的 CPU 进行授时 - 给GEAC控制器外部的传感器（例如雷达）进行授时
4	产生触发脉冲给到相机	产生触发脉冲给到相机，并且记录触发信号上升沿时刻的时间戳以通信的方式给到 CPU

Main functions of CPLD

Feature	Function description	Functional supplement
1	Communicates with internal cpus over serial ports	CPLD parses the CPU instruction set and performs the corresponding logic control function
2	Support multiple ways to time yourself	- The CPU uses its system time to time the CPLD - Internal GPS chip timing CPLD - CPLD timing by external inertial navigation equipment
3	CPLD provides external time	- Timing the CPU inside the GEAC controller - Timing sensors (e.g. radar) outside the GEAC controller

4	Generates a trigger pulse to the camera	The generated trigger pulse is given to the camera, and the time stamp recording the rising edge time of the trigger signal is given to the CPU in a communication manner
---	---	---

特别注明

specific description

CPU 同 CPLD 间的通信串口：

接口	软件接口	波特率	功能说明
UART(CPU 与 CPLD 内部直连)	/dev/ttyTHS1	115200	配置 CPLD 功能

Serial port for communication between CPU and CPLD:

port	Software interface	Baud rate	Function description
UART(Direct connection between the CPU and the CPLD)	/dev/ttyTHS1	115200	Configure the CPLD function

CPU 从 CPLD 获取 GPRMC 数据串口：

接口	软件接口	波特率	功能说明
UART(CPU 与 CPLD 内部直连)	/dev/ttyTHS4	默认9600	CPLD 输出 GPRMC 给 CPU

The CPU obtains the GPRMC data serial port from the CPLD:

port	Software interface	Baud rate	Function description
UART(Direct connection between the CPU and the CPLD)	/dev/ttyTHS4	Default 9600	The CPLD outputs GPRMC to the CPU

CPU 配置内部 GPS 模块串口：

接口	软件接口	波特率	功能说明
UART(CPU 与 CPLD 内部直连)	/dev/ttyTHS4	9600	CPU 配置 GPS 模块串口，通过 CPLD 透传 GPS 信号到/dev/ttyTHS4

CPU configuration internal GPS module Serial port:

port	Software interface	Baud rate	Function description
UART(Direct connection between the CPU and the CPLD)	/dev/ttyTHS4	9600	The serial port of the GPS module is configured on the CPU, and the GPS signal is transparently transmitted to /dev/ttyTHS4 through the CPLD

CPLD 固件版本查询及升级

Querying and upgrading the CPLD firmware version

工具获取 (Tool acquisition) :

- [tztek-jetson-tool-cpld-get-info-510jx0-v1.0.deb](#)

#安装工具包，获取信息 (Install the tool package and obtain information)

```
sudo dpkg -i tztek-jetson-tool-cpld-get-info-510jx0-v1.0.deb
```

#获取CPLD版本信息 (Obtain the CPLD version information)

```
sudo tztek-jetson-tool-cpld-get-info-510jx0
```

CPLD 功能详细描述

This section describes the CPLD functions

串口通信

本通信协议基于串口，遵循握手机制，上位机（CPU）主要依靠该协议对 CPLD 内部的寄存器进行读写，以及对特定的数据流（时间戳）进行读写。

Serial communication

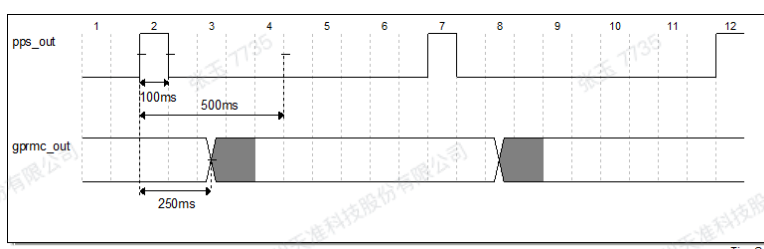
This communication protocol is based on the serial port and follows the handshake mechanism. The host computer (CPU) mainly relies on this protocol to read and write registers in CPLD and to read and write specific data streams (time stamps).

同步授时

在进行同步授时时，CPLD 会发送一个名为 PPS 的脉冲信号，其脉宽，周期分别固定为100ms，1s；以及利用串口发送 GPRMC 数据包，输出频率固定为1hz，波特率支持 9600bps 与 115200bps 两种（可通过寄存器配置）。同步授时接口时序如下图所示：

Synchronous timing

During synchronous timing, CPLD will send a pulse signal named PPS, whose pulse width and period are fixed to 100ms and 1s respectively. And the use of serial port to send GPRMC data packets, the output frequency is fixed at 1hz, the baud rate supports 9600bps and 115200bps two (can be configured through registers). The timing of the synchronous timing interface is shown in the figure below:



本产品会在距每个 PPS 上升沿 250ms 处开始发送一帧 GPRMC 数据包，能确保在距 PPS 上升沿500ms内发送完毕。

The product will start sending a GPRMC packet at 250ms from the rising edge of each PPS, ensuring that the transmission is completed within 500ms from the rising edge of the PPS.

同步触发

CPLD 解析 CPU 发送的指令集并据此获得相应的触发参数（触发间隔，触发正脉宽，触发相位差）来生成触发信号，并将触发信号发送至GMSL相机，在产生触发的同时 CPLD 会记录下实时的触发时间戳并将其发送给 CPU（走通信协议）。

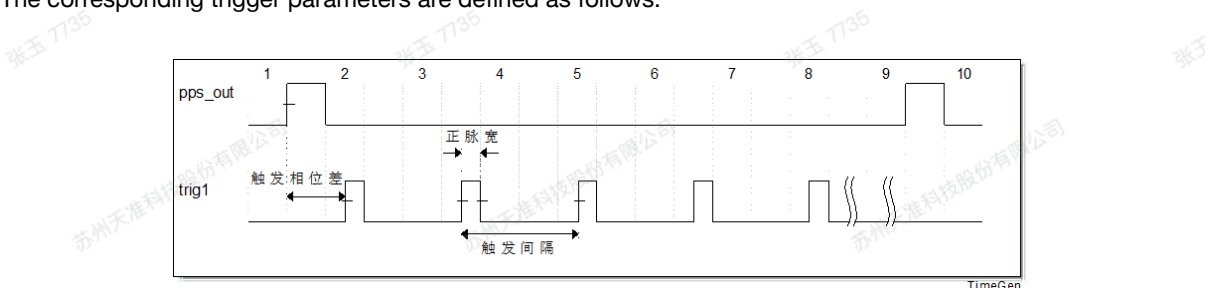
Synchronous trigger

CPLD analyzes the instruction set sent by the CPU and obtains the corresponding trigger parameters (trigger interval, trigger positive pulse width, trigger phase difference) to generate the trigger signal, and sends the trigger signal to the

GMSL camera. At the same time, CPLD records the real-time trigger time stamp and sends it to the CPU (go communication protocol).

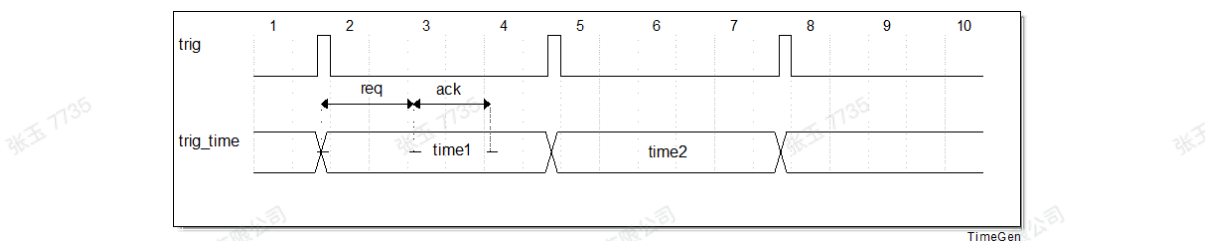
相应触发参数的定义如下图所示：

The corresponding trigger parameters are defined as follows:



CPLD 在触发信号的上升沿将当前的触发时间戳寄存下来，CPU 收到触发上升沿后便向 CPLD 发送读触发时间戳的请求，CPLD 收到请求后将触发时间戳返回给 CPU，整个交互过程必须在一个触发间隔内结束，如下图所示：

CPLD stores the current trigger timestamp at the rising edge of the trigger signal. After receiving the trigger rising edge, the CPU sends a request to the CPLD to read the trigger timestamp, and the CPLD returns the trigger timestamp to the CPU after receiving the request. The whole interaction process must be completed within a trigger interval, as shown in the following figure:



技术指标

Technical index

触发信号

触发间隔、正脉宽持续时间、相位偏移参数均可独立设置。

精度：由于 CPLD 时钟晶振频率为 100Mhz，故精度可达 10ns 级。

Trigger signal

Trigger interval, positive pulse width duration, and phase offset parameters can be set independently.

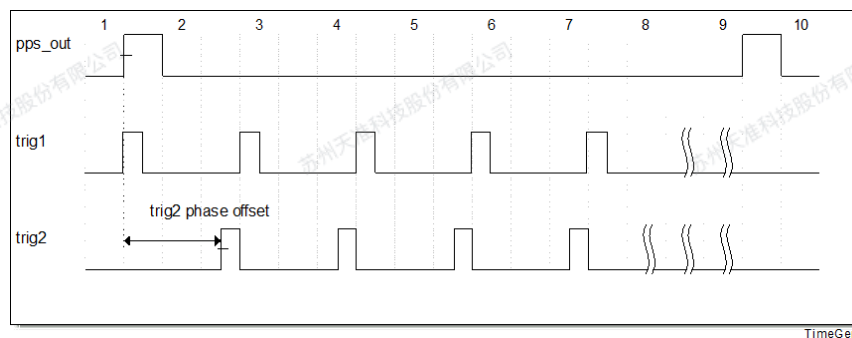
Accuracy: Because the CPLD clock crystal frequency is 100Mhz, the accuracy can reach 10ns level.

触发时间戳

CPLD 会产生多个触发信号，并支持在各触发信号间设置偏移量，如下图所示：（下图中所有触发的周期、脉宽均相同，仅是每路触发的偏移量设置不同）

Trigger timestamp

CPLD generates multiple trigger signals, and supports setting offsets between each trigger signal, as shown in the following figure: (All trigger cycles and pulse widths are the same in the following figure, but the offset Settings of each trigger are different)



而 trig1 为第一路触发，其自身不带偏移量，因此 CPLD 将选用 trig1 作为基准触发信号，并选择将其对应的触发时间戳经通信串口上传给 CPU。后续几路触发对应的时间戳可由 CPU 在上层基于第一路触发时间戳直接加上偏移量计算得到。

Since trig1 is the first trigger without offset, CPLD selects trig1 as the benchmark trigger signal and sends its corresponding trigger time stamp to the CPU through the communication serial port. The time stamps corresponding to subsequent triggers can be calculated by the CPU on the upper layer based on the first trigger time stamp and directly add the offset.

注意：触发时间戳的时间来源分为两类：若为室内 CPU 授时，则时间源为 CPU 的系统时间；若为室外第三方设备授时，则时间源为 GPS 卫星的时间。

频率：与 trig1 的频率一致，并且无明显跳变（例如时间戳回退）。

精度：可精确到小数点后8位（0.0000000x）。

Note: There are two types of time sources that trigger the timestamp: If the indoor CPU is timing, the time source is the system time of the CPU; If it is an outdoor third-party device, the time source is the time of the GPS satellite.

Frequency: Consistent with trig1's frequency, with no significant jump (e.g. timestamp backtracking).

Accuracy: Accurate to 8 decimal places (0.0000000x).

CPU 授时

CPU timing

工作原理：当处于室内或是无卫星信号的环境下，CPU 可以经通信串口给 CPLD 发送系统时间戳以完成授时。

Working principle: When in the indoor or no satellite signal environment, the CPU can send the system time stamp to the CPLD through the communication serial port to complete the timing.

注意：CPU 不会给 CPLD 提供 PPS 信号，只给 CPLD 发送系统时间。CPLD 输出的 GPRMC 数据包中的时间戳每秒更新一次，无明显跳变（例如时间戳回退）。

Note: The CPU does not provide a PPS signal to the CPLD, only the system time to the CPLD. The timestamp in the GPRMC packet output by the CPLD is updated once per second without significant change (such as timestamp backtracking).

外部第三方设备授时

External third-party devices provide time

一般情况下，用户均会接上天线靠 GPS 设备给 GEAC 系统进行授时。

其中第三方设备有两种：

- 机盒内部的中科微芯片
- 机盒外部的惯导

CPU 可以通过给 CPLD 发送相应指令进行手动切换，第三方设备需要给 CPLD 提供 PPS，GPRMC 信号。

In general, the user will attach an antenna and use a GPS device to give time to the GEAC system.

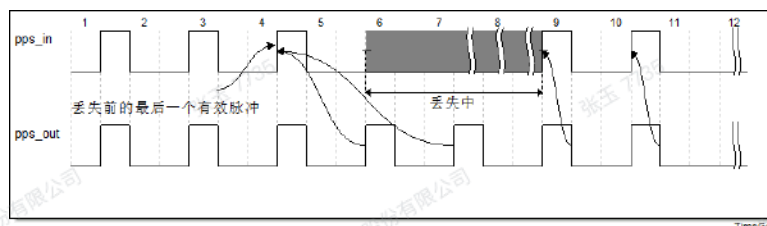
There are two types of third-party devices:

- Zhongke microchip inside the box
- Inertial navigation outside the box

The CPU can manually switch by sending corresponding commands to the CPLD, and third-party devices need to provide PPS and GPRMC signals to the CPLD.

当第三方设备正常提供 PPS 信号时，CPLD 输出的 PPS 信号会与其保持一致；当第三方设备由于其他原因导致 PPS 信号丢失时，CPLD 依然会保持输出 PPS 信号，并且此时 PPS 的周期与第三方设备丢失前的最后一个有效的 PPS 信号一致；当第三方设备的 PPS 信号后期恢复时，CPLD 输出的 PPS 信号会立即与其保持一致。如下图所示：

When a third-party device normally provides PPS signals, the CPLD outputs the same PPS signals. When a third-party device loses the PPS signal due to other reasons, the CPLD still outputs the PPS signal, and the PPS period is the same as the last effective PPS signal before the third-party device loses the PPS signal. When the PPS signal of a third-party device is recovered later, the PPS signal output by the CPLD is the same as that of the third-party device. As shown in the picture below:



CPLD 产生的 PPS 信号的周期误差在 $\pm 2\mu\text{s}$ 内，误差是由于 CPLD 晶振受到干扰不稳定所致。

当第三方设备提供的 GPRMC 数据包中的时间戳字段不缺失时，CPLD 便认为该时间戳有效，并直接将其输出；当提供的 GPRMC 数据包中时间戳字段缺失且状态字段值为“V”，则 CPLD 会在自己维护的时间戳的基础上每秒累加1次输出。

The period error of PPS signal generated by CPLD is within $\pm 2\mu\text{s}$, and the error is caused by the instability of CPLD crystal.

When the timestamp field in the GPRMC packet provided by a third-party device is not missing, CPLD considers the timestamp valid and outputs it directly. When the timestamp field is missing in the supplied

GPRMC packet and the status field value is "V", the CPLD will add up the output once per second on the basis of the self-maintained timestamp.

具体使用

Specific use

GPS 授时

可接收内部和外部的 GPS 信号，对内部 CPU 系统授时或者对外部的需要授时的设备进行授时（比如激光雷达和IMU），具体操作可参考 GPS 的配置和使用。

GPS timing

It can receive internal and external GPS signals, time the internal CPU system or time the external device that needs to time (such as Lidar and IMU), the specific operation can refer to the configuration and use of GPS.

外部惯导第三方授时

本产品支持具备 RS232 的 GPRMC 、 PPS 信号的外部惯导给 CPLD 授时。

External inertial navigation third party timing

This product supports the external inertial navigation system with RS232 GPRMC and PPS signals to CPLD timing

外部惯导 GPRMC 选择RS232

切换为外部惯导授时时，默认选择为 RS232 电平的 GPRMC 信号，波特率 115200bps，PPS 为 TTL 电平。

External inertial navigation GPRMC selects RS232

When switching to external inertial navigation time, GPRMC signal at RS232 level is selected by default, baud rate is 115200bps, PPS is TTL level.

GPRMC 信号输入使用 **D1** 引脚，PPS - TTL 信号输入使用 **J2** 引脚。

接口名称	引脚序号	接口信号定义	备注
RS232- GPRMC	D1	RS232_RX_GPRMC_CON	可通过配置串口，配置CPLD接受外部惯导输入的GPS的 GPRMC信号，232电平，波特率115200bps

PPS	J2	PPS_IN2	可通过配置串口，使CPLD接受外部惯导输入的PPS信号，TTL电平，默认接收惯导PPS输入
-----	----	---------	---

GPRMC signal input uses D1 pin, PPS-TTL signal input uses J2 pin.

Interface name	Pin number	Interface signal definition	remark
RS232-GPRMC	D1	RS232_RX_GPRMC_CON	By configuring the serial port, the CPLD can be configured to accept the external inertial navigation input GPS GPRMC signal, 232 level, baud rate 115200bps
PPS	J2	PPS_IN2	You can configure the serial port to enable the CPLD to accept the external inertial navigation device (INS) input PPS signal, TTL level, and receive the INS PPS input by default

1、推荐使用CPLD-TEST工具，具体使用方法可参考以下链接（需要安装CPLD固件）

1. It is recommended to use the CPLD-test tool. For details, see the following link (CPLD firmware needs to be installed).

[CPLD-TEST工具使用说明](#)

2、使用外部惯导源的 RS232 接入 GAECX1 设备的接口引脚，按照下方的两个 python 脚本把 CPLD 授时模式切换为外部惯导源。（脚本需使用python3环境）

2. Use the RS232 of the external inertial navigation source to access the interface pin of the GAECX1 device, and switch the CPLD timing mode to the external inertial navigation source according to the two python scripts below. (The script needs to use the python3 environment)

[510g_r3_0_external_time_service.py](#)

[510g_r3_0_external_time_service_RS232.py](#)

操作 (operation)	命令 (command)
切换为使用外部惯导授时源	sudo python 510g_r3_0_external_time_service.py

Switch to using an external inertial navigation timing source	
外部惯导授时源使用RS232电平的 GPRMC	
The external inertial navigation timing source uses GPRMC at RS232 level	<code>sudo python 510g_r3_0_external_time_service_RS232.py</code>

切换后默认 bitrate 为115200，如需要切换9600波特率，可修改脚本把对应配置9600通信的协议码下发配置。

The default bitrate is 115200. If you want to change the baud rate to 9600 baud rate, you can modify the script to deliver the protocol code corresponding to the 9600 communication configuration.

脚本文件如下（The script file is as follows）：

[510g_r3_0_external_time_service_RS232.py](#)

注意：惯导验证方法及效果参考 GPS 授时功能介绍。

Note: Refer to GPS timing function introduction for the methods and effects of inertial navigation verification.

GPS

简介

本机器自带 GPS 功能。GPS 接口外接信号采集设备，比如直接接一个蘑菇头，就可以通过 GPS 接口将位置信息以及时间信息传输到机器内部自带的 GPS 芯片进行处理。GPS 接口位置可参考接口介绍，利用该 GPIO 口，客户可以外接自行选择的合适的 GPS 设备。

Introduction

This machine comes with GPS function. The GPS interface is connected to the external signal acquisition device, such as directly connecting a mushroom head, and the position information and time information can be transmitted to the GPS chip inside the machine for processing through the GPS interface. For the GPS interface location, refer to the interface description. Using this GPIO port, customers can connect the appropriate GPS device of their own choice.

接线（Wiring connection）

外接信号采集设备（GPS 天线）

本产品内置 GPS 模块，GPS 接口可直接外接信号采集设备，如 GPS 天线。

External signal acquisition equipment (GPS antenna)

This product has built-in GPS module, GPS interface can directly connect to external signal acquisition equipment, such as GPS antenna.



GPS 的配置（GPS configuration）

使用 CPU 配置串口参数验证内置 GPS 芯片（Verify the built-in GPS chip using CPU configuration serial port parameters）

- [tztek-jetson-tool- GPS -flash-atgm332d-v2.0.deb](#)

```
#安装deb包（Installing the deb package）
```

```
sudo dpkg -i tztek-jetson-tool-gps-flash-atgm332d-v2.0.deb
```

```
#使用（Use）
```

```
sudo tztek-jetson-tool-gps-flash-atgm332d
```

GPS使用PGPS usage） 使用

硬件使用 GPS 数据进行时钟的同步，一般需要安装chrony、gpsd工具集等工具。

GEACX1使用 CPLD 提供 GPRMC 数据信号分别发给 CPU，发送 GPRMC 和 PPS 用于 gpsd和chrony的锁定。

The hardware uses GPS data to synchronize clocks. Therefore, tools such as chrony and gpsd tools need to be installed.

GEACX1 uses the CPLD to provide GPRMC data signals to the CPU, and sends GPRMC and PPS for gpsd and chrony locking.

安装时间同步服务器chrony (Install the time synchronization server chrony)

```
#安装chrony (Install chrony)

sudo apt-get install chrony

#重启chrony服务器，使chrony生效 (Restart the chrony server for chrony to take effect)

service chrony restart
```

重启chrony服务器，提示需要输入用户密码，点击授权Authenticate后，服务生效

After the chrony server is restarted, a message is displayed indicating that you need to enter the user password. After you click Authorization Authenticate, the service takes effect

安装 gpsd工具集

下载配置系统授时同步工具 [orin-gps-tools-v2.0.zip](#)，修改配置文件后执行脚本安装，具体步骤参考如下：

Install the gpsd toolset

Download and configure the system time synchronization tool [orin-gps-tools-v2.0.zip](#) , modify the configuration file, and then execute the installation script. For details, see the following steps:

```
#解压文件 (Unzip file)

unzip gpsd_config.zip

# 调转解压后的目录中 (Go to the decompressed directory)

cd gpsd_config

#赋予脚本执行权限 (Grant execution permission to the script)

chmod +x gpsd.sh

#赋予脚本执行权限 (Grant execution permission to the script)

chmod +x gpsd.sh

#安装 (Install)
```

```
sudo ./gpsd.sh
```

压缩包中的gpsd的配置文件已经写好，对应的接收串口均为/dev/ttyTHS1 bitrate: 9600

The gpsd configuration

file in the compressed package has been written, and the corresponding receiving serial port is /dev/ttyTHS1 bitrate: 9600

查看 GPS 输出

View GPS output

方式一：安装完成后执行 `cgps -s` 查看是否有 GPS 数据

Method 1: After the installation, run the `cgps -s` command to check whether GPS data exists

```
Time: 2020-01-01T00:28:33.000Z PRN: Elev: Azim: SNR: Used:
Latitude: n/a
Longitude: n/a
Altitude: n/a
Speed: 0.00 kph
Heading: 0.0 deg (true)
Climb: n/a
Status: 2D FIX (9 secs)
Longitude Err: n/a
Latitude Err: n/a
Altitude Err: n/a
Course Err: n/a
Speed Err: n/a
Time offset: 0.002
Grid Square: n/a
```

如图是在室内所测，没有 GPS 信号，时间默认从2020年1月1日00:00开始计时。

外接 GPS，GPS 放到室外后，若有 GPS 信号的情况下，Time 为当前时间，正确结果如下图：

The figure is measured indoors without GPS signal, and the time starts from 00:00 on January 1, 2020 by default.

After the GPS is placed outdoors, if there is a GPS signal, the Time is the current time. The correct result is shown in the following figure

```
Time: 2021-12-28T07:23:45.000Z PRN: Elev: Azim: SNR: Used:
Latitude: 37.40
Longitude: 121.40
Altitude: n/a
Speed: 0.00 kph
Heading: 299.7 deg (true)
Climb: n/a
Status: 2D FIX (8 secs)
Longitude Err: n/a
Latitude Err: n/a
Altitude Err: n/a
Course Err: n/a
Speed Err: n/a
Time offset: 0.334
Grid Square: PM01fi
```

方式二：终端输入 `gpsmon`，若有 GPS 信号，会显示当前时间和 GPRMC 信息。

Method 2: Enter `gpsmon` on the terminal. If there is a GPS signal, the current time and GPRMC information are displayed.

验证授时是否成功 (Verify that the timing is successful)

在终端执行 `watch chronyc sources -v` 会出现如下图所示结果

Run `watch chronyc sources -v on the terminal`. The following figure shows the result

红色框内的 GPSD 前有 * 标志表示接收到 GPS 信号

PPS 信号前有 ? 表示 PPS 没有锁存, 不过可以使用其他方式来查看 PPS 状态

在终端内输入 `dmesg` 可以出现如下结果:

“*” in front of the GPSD in the red box indicates that a GPS signal has been received

Before the PPS signal? The PPS does not have a latch. However, you can use other methods to view the PPS status

Enter `dmesg` in the terminal to get the following result:

```

nvidia@nvidia-desktop:~$ dmesg
[ 0.000000] Booting Linux on physical CPU 0x0
[ 0.000000] Linux version 4.9.140-tegra (tztek@build) (gcc version 7.3.1 20180425
[linaro-7.3-2018.05 revision d29120a424ecfbc167ef90065c0eeb7f91977701] (Linaro GCC
7.3-2018.05) ) #13 SMP PREEMPT Mon Oct 12 19:39:29 CST 2020 tztek_version:v0.0.0_510
g_r2_1_debug
[ 0.000000] Boot CPU: AArch64 Processor [4e0f0040]
[ 0.000000] OF: fdt:memory scan node memory, reg size 48,
[ 0.000000] OF: fdt: - 00000000 , 2c000000
[ 0.000000] OF: fdt: - ac200000 , 44600000
[ 0.000000] OF: fdt: - 10000000 , 78000000
[ 0.000000] earlycon: tegra_comb_uart0 at MMIO32 0x00000000c168000 (options '')
[ 0.000000] bootconsole [tegra_comb_uart0] enabled
[ 0.000000] Found tegra_fbmem: 00800000a069d000
[ 0.000000] Found lut_mem: 00002000a0697000
[ 0.000000] OF: fdt:Reserved memory: failed to reserve memory for node 'fb1_carve
out': base 0x0000000000000000, size 0 MiB
[ 0.000000] OF: fdt:Reserved memory: failed to reserve memory for node 'fb1_carve
out': base 0x0000000000000000, size 0 MiB
[ 0.000000] OF: fdt:Reserved memory: failed to reserve memory for node 'fb2_carve
out': base 0x0000000000000000, size 0 MiB
[ 0.000000] OF: fdt:Reserved memory: failed to reserve memory for node 'fb2_carve

324.505860] pps pps0: PPS_FETCH
324.505878] pps pps0: timeout 0.000000000
325.116227] pps pps0: PPS event at 1577837122.703975664
325.116242] pps pps0: capture assert seq #323
325.506947] pps pps0: PPS_FETCH
325.506966] pps pps0: timeout 0.000000000
326.116172] pps pps0: PPS event at 1577837123.703972277
326.116185] pps pps0: capture assert seq #324
326.507994] pps pps0: PPS_FETCH
326.508012] pps pps0: timeout 0.000000000
327.116123] pps pps0: PPS event at 1577837124.703973307
327.116137] pps pps0: capture assert seq #325
327.509047] pps pps0: PPS_FETCH
327.509067] pps pps0: timeout 0.000000000
328.116072] pps pps0: PPS event at 1577837125.703974272
328.116085] pps pps0: capture assert seq #326
328.510105] pps pps0: PPS_FETCH
328.510123] pps pps0: timeout 0.000000000
329.116022] pps pps0: PPS event at 1577837126.703976317
329.116038] pps pps0: capture assert seq #327
329.511166] pps pps0: PPS_FETCH
329.511183] pps pps0: timeout 0.000000000
nvidia@nvidia-desktop:~$

```

可以看到有 PPS 的触发信号，红色框内为 PPS 序列号，蓝色框为 PPS 时间戳。

注意：若在执行 `watch chronyc sources -v` 后出现如下情况

The PPS trigger signal is displayed. The red box indicates the PPS serial number, and the blue box indicates the PPS time timestamp.

Note: If the following situation occurs after executing `watch chronyc sources -v`

```

Every 2.0s: chronyc sources -v          nvidia-desktop: Thu Nov 12 15:07:43 2020
506 Cannot talk to daemon

```

则说明 chrony 服务没有生效，在终端重新执行 `service chrony restart` 后再执行

`watch chronyc sources -v` 后再进行尝试，正确结果如下图：

The chrony service does not take effect. Run the `service chrony restart` command again on the terminal

`watch chronyc sources -v` and then try again, the correct result is as follows:

```
Every 1.0s: chronyc sources -v
210 Number of sources = 2

--- Source mode '^' = server, '=' = peer, '#' = local clock.
/.. Source state '*' = current synced, '+' = combined , '-' = not combined,
/.. '?' = unreachable, 'x' = time may be in error, '~' = time too variable.

Reachability register (octal) -.      xxxx [ yyyy ] +/- zzzz
Log2(Polling interval) --.          xxxx = adjusted offset,
                                yyyy = measured offset,
                                zzzz = estimated error.

MS Name/IP address      Stratum Poll Reach LastRx Last sample
=====
## GPS                  0 4 377 12 +298ns[ +529ns] +/- 474ns
#? NMEA                 0 4 377 14 -167ms[ -167ms] +/- 100ms
```

故障排查 (troubleshooting)

Q1: 授时失败

A1: 先查看 GPS 是否有输出

`sudo cat /dev/ttyTHS1`, 检查打印出的 GPRMC 数据是否正常。

Q1: Time service failed

A1: First check whether GPS has output

`Sudo cat/dev/ttyTHS1`, to check if the printed GPRMC data is normal.

```
nvidia@nvidia-desktop:~$ sudo cat /dev/ttyTHS1
$GPRMC,004749.00,V,,0,,0,00000,,010120,,0,A*7E

$GPRMC,004750.00,V,,0,,0,00000,,010120,,0,A*76

$GPRMC,004751.00,V,,0,,0,00000,,010120,,0,A*77

$GPRMC,004752.00,V,,0,,0,00000,,010120,,0,A*74

$GPRMC,004753.00,V,,0,,0,00000,,010120,,0,A*75

$GPRMC,004754.00,V,,0,,0,00000,,010120,,0,A*72

$GPRMC,004755.00,V,,0,,0,00000,,010120,,0,A*73

$GPRMC,004756.00,V,,0,,0,00000,,010120,,0,A*70
```

再查看 GPS 的 PPS 信号是否有输出, `sudo echo debug > /proc/loglevel && dmesg -w`

直接查看对应的刷新 PPS 时间戳是否一秒一次

To check whether the PPS signal of the GPS is output, run `sudo echo debug > /proc/loglevel &&dmesg -w`

Check whether the corresponding PPS timestamp is refreshed once a second

```
1195.001295] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
1195.001191] t1rk debug: /home/xiaolai/work/code/mvtda/platform/code/source/kernel_src/kernel/kernel-4.9/drivers/ppp/kapi.c~100: PPS event at 1040165746.931128811
```

```
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->288: pps_event at 1640165746.93111881
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->211: capture assert seq #2497
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->288: pps_event at 1640165749.93111977
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->211: capture assert seq #2498
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->288: pps_event at 1640165746.93114445
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->211: capture assert seq #2499
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->288: pps_event at 1640165747.93114423
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->211: capture assert seq #2500
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->288: pps_event at 1640165748.93115178
kernel/kernel-4.9/drivers/ppp/kapi.c - pps_event ->211: capture assert seq #2501
```

CAN

GEACX1 使用 NVIDIA ORIN 核心板，包含两路 nvidia 原生 CAN 控制器。本产品 CAN 总线支持 CAN 和 CANFD 通信协议，CANFD 是 CAN 协议的升级版，主要体现在传输速率不同、数据长度不同、帧格式不同、ID 长度不同。接线时，高接高，低接低，建议客户将地线接上，外设不用接终端电阻（机器内部已装配了终端电阻，为120Ω）。根据传输线理论，终端电阻可以吸收网络上的反射波，有效地增强信号强度。CAN 设备协议为标准 socket CAN 支持网络编程。

CAN

The GEACX1 uses the NVIDIA ORIN core board and contains two nvidia native CAN controllers. The CAN bus of this product supports CAN and CANFD communication protocols. CANFD is an upgraded version of CAN protocol, which is mainly reflected in different transmission rates, different data lengths, different frame formats, and different ID lengths. When wiring, high connect high, low connect low, it is recommended that the customer connect the ground wire, the peripheral does not need to connect the terminal resistor (the machine has been equipped with a terminal resistor, 120Ω). According to the transmission line theory, the terminal resistance can absorb the reflected waves on the network, effectively enhancing the signal strength. The CAN device protocol supports network programming for standard socket CAN.

CAN配置方法（CAN configuration method）

设置波特率

支持的波特率如下表：

	canfd	can2.0波特率（bps）	canfd数据段波特率（bps）
can0	支持	500K, 1M	1M,2M,3M
can1	支持	500K, 1M	1M,2M,3M

Set baud rate

The following table lists the supported baud rates:

	canfd	can2.0 Baud Rate (bps)	canfd Data Segment Baud Rate (bps)
can0	support	500K, 1M	1M,2M,3M

can1	support	500K, 1M	1M,2M,3M
------	---------	----------	----------

默认波特率是500k，可通过命令更改。更改波特率之前，需要先关闭 can，再进行波特率的设置。

The default baud rate is 500k, which can be changed by command. Before changing the baud rate, you need to disable can before setting the baud rate.

#ifconfig查看can信息 (ifconfig Displays can information)

ifconfig -a

#关闭can0 (Close can0)

sudo ip link set down can0

#设置can0波特率为500Kbps (Set the baud rate of can0 to 500Kbps)

sudo ip link set can0 type can bitrate 500000

#打开can0 (Open can0)

sudo ip link set up can0

can 打开后，[ifconfig -a](#) 可以查看到 can 信息：

After can is enabled, ifconfig -a displays can information:

```
nvidia@nvidia-desktop:~/Downloads$ ifconfig -a
can0: flags=128<NOARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 129

can1: flags=128<NOARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 130

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:51:ff:7b:8d txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
```

can发送数据

两种方式发送数据

使用 `cansend` 命令发送数据

can send data

Two ways to send data

Send data using the `cansend` command

```
nvidia@nvidia-desktop: ~  
nvidia@nvidia-desktop:~$ sudo ip link set can1 down  
nvidia@nvidia-desktop:~$ sudo ip link set can1 type can bitrate 500000  
nvidia@nvidia-desktop:~$ sudo ip link set can1 up  
nvidia@nvidia-desktop:~$ cansend can1 123#DEADBEEF  
nvidia@nvidia-desktop:~$
```

使用 `cangen` 命令发送数据 (Send data using the `cangen` command)

```
nvidia@nvidia-desktop: ~  
nvidia@nvidia-desktop:~$ sudo ip link set can1 down  
nvidia@nvidia-desktop:~$ sudo ip link set can1 type can bitrate 500000  
nvidia@nvidia-desktop:~$ sudo ip link set can1 up  
nvidia@nvidia-desktop:~$ cangen can1  
nvidia@nvidia-desktop:~$
```

使用 `candump` 命令接收数据 (Use the `candump` command to receive data)

```
nvidia@nvidia-desktop: ~  
nvidia@nvidia-desktop:~$ sudo ip link set can1 down  
nvidia@nvidia-desktop:~$ sudo ip link set can1 type can bitrate 500000  
nvidia@nvidia-desktop:~$ sudo ip link set can1 up  
nvidia@nvidia-desktop:~$ candump can1  
can1 000 [8] 00 01 02 03 04 05 06 07  
can1 000 [8] 00 01 02 03 04 05 06 07  
can1 000 [8] 00 01 02 03 04 05 06 07  
can1 000 [8] 00 01 02 03 04 05 06 07  
nvidia@nvidia-desktop:~$
```

canfd 配置方法 (canfd configuration method)

设置can0为canfd模式，波特率500Kbps，数据端波特率2Mbps (Set can0 to canfd mode with a baud rate of 500Kbps and a baud rate of 2Mbps on the data side)

```
sudo ip link set can0 type can bitrate 500000 dbitrte 2000000 fd on
```

canfd 和 can 2.0 的区别：其中 bitrate 为 can2.0 模式下的波特率；dbitrte can fd模式下的波特率。根据官方文档，这个值最大可配置为5M，想要实际在工况下稳定可靠传输，采用2M左右比较合理。

The differences between canfd and can2.0 are as follows: bitrate is the baud rate in can2.0 mode; dbitrates can be baud rate in fd mode. According to official documents, this value can be configured to a maximum of 5M, in order to actually stable and reliable transmission under working conditions, the use of about 2M is more reasonable.

canfd 发送数据 (canfd sends data)

```
nvidia@nvidia-desktop: ~  
nvidia@nvidia-desktop:~$ sudo ip link set can1 down  
nvidia@nvidia-desktop:~$ sudo ip link set can1 type can bitrate 500000 dbitrates 2000000 fd on  
nvidia@nvidia-desktop:~$ sudo ip link set can1 up  
nvidia@nvidia-desktop:~$ cangen -b can1
```

canfd 接收数据 (canfd accepts data)

```
nvidia@nvidia-desktop: ~  
nvidia@nvidia-desktop:~$ sudo ip link set can1 down  
nvidia@nvidia-desktop:~$ sudo ip link set can1 type can bitrate 500000  
nvidia@nvidia-desktop:~$ sudo ip link set can1 up  
nvidia@nvidia-desktop:~$ candump can1  
can1 000 [8] 00 01 02 03 04 05 06 07  
can1 000 [8] 00 01 02 03 04 05 06 07  
can1 000 [8] 00 01 02 03 04 05 06 07  
can1 000 [8] 00 01 02 03 04 05 06 07
```

示例：使用 can-utils 小工具进行测试 (Example: Testing with the can-utils gadget)

can0发送can2.0报文 (can0 sends the can2.0 packet)

cansend can0 123#DEADBEEF

can0发送canfd报文 (can0 Sends canfd packets)

cansend can0 123##112345678

can0发送随机can2.0报文 (can0 sends random can2.0 packets)

cangen can0

can0发送随机canfd报文 (can0 Sends random canfd packets)

cangen -b can0

can0打印接收到的报文 (can0 prints the received packet)

```
candump can0
```

更多命令参考官网 <https://elinux.org/Can-utils>

For more commands, see the official website: <https://elinux.org/Can-utils>

can 使用 (can usr)

使用 can 分析仪测试数据。

can-L 连接 can-L, can-H 连接 can-H; 建议将 GND 接上; 外设不用接终端电阻 (机器内部的 can 硬件已经安装了 120OHM 的终端电阻)。

Test data using can analyzer.

can-L connects to can-L, can-H connects to can-H; It is recommended that GND be connected; The peripheral does not need to be connected to a terminal resistor (a 120OHM terminal resistor is already installed in the can hardware inside the machine).

```
#安装canutils (Install canutils)

sudo apt install canutils

# can0 接收数据 (can0 receives data)

candump can0
```

如果不打印出 can 数据, 就对接收 can 接口重新使能:

If you do not print out can data, re-enable the receiving can interface:

```
sudo ip link set down can0

sudo ip link set can0 type can bitrate 500000

sudo ip link set up can0

candump can0
```

故障排查 (troubleshooting)

Q1: candump 无数据打印

A1: `ifconfig -a` 查看 can 状态。

Q1: candump has no data to print

A1: `ifconfig -a` Displays the can status.

```
nvidia@nvidia-desktop:~$ ifconfig -a
can0: flags=128<N0ARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 62
```

如果此时 `candump` 数据打印正常，检查外部线束是否连通。

If `candump` data is printed correctly, check whether the external wiring harness is connected.

Q2: can0 的使用，报文发送后，`candump` 很久才接收到，或者接收不到

A2: 检查总线 CAN_H 和 CAN_L 物理连接是否松动或者接反，建议接地和移除终端电阻（机器内部已有）。

Q2: can0 is used. `candump` receives or fails to receive a packet after it is sent

A2: Check whether the physical connection between CAN_H and CAN_L is loose or reversed. It is recommended to ground and remove the terminal resistor (already in the machine).

Q3: can0 的使用，`cansend` 发送报文，返回 “No buffer space available”

A3: 默认 `txqueuelen=10`，所以 can 发送到总线超过10个报文未被接收后，无法再发送报文，需要等到报文被接收后才能再次发送报文。选择增加 `txqueuelen` 值，设置后重启机器。该问题常见原因与问题1相同。具体命令如下：

Q3: can0 is used. `cansend` sends a packet and returns "No buffer space available".

A3: `txqueuelen=10` by default. Therefore, if more than 10 packets can be sent to the bus but are not received, packets cannot be sent again and can be sent only after packets are received. Select increase `txqueuelen` and restart the machine. The common cause of this problem is the same as in question 1. Specific commands are as follows:

```
# 获取所有can设备的txqueuelen接口 (Gets the txqueuelen interface for all can devices)

sudo find /sys/device/ -name "tx_queue_len" | grep "can"
```

```
# 输出结果can0, can1, can2 (输出结果can0, can1, can2)

/sys/devices/c310000.mttcan/net/can0/tx_queue_len

/sys/devices/c320000.mttcan/net/can1/tx_queue_len

/sys/devices/c260000.spi/spi_master/spi1/spi1.0/net/can2/tx_queue_len

# 添加can0的txqueuelen接口的写权限 (Add write permission for the txqueuelen interface of can0)

sudo chmod 666 /sys/devices/c310000.mttcan/net/can0/tx_queue_len

# 设置can0的txqueuelen=1000 (Set txqueuelen=1000 for can0)

sudo echo 1000 > /sys/devices/c310000.mttcan/net/can0/tx_queue_len
```

UART

串口种类及区别

串口协议类型	RS232	RS485
工作模式	全双工	全双工
传输距离(max)	15m	1200m
节点	1收~1发	1收~1发
传输速率 (max)	200kb/s	10Mb/s
信号线数量	3根 (GND/TXD/RXD)	2根 (A/B)

Serial port types and differences

Serial protocol type	RS232	RS485
Working mode	Full duplex	Full duplex
Transmission distance (max)	15m	1200m
node	1 Receive to 1 send	1 Receive to 1 send

Transfer Rate (max)	200kb/s	10Mb/s
Number of signal lines	3 roots (GND/TXD/RXD)	2 roots (A/B)

PIN串口使用 (PIN serial port usage)

引脚定义

引脚序号	引脚信号定义	软件设备名
J3	RS485_2_A1_CON_2_RS232_1_TX_R	/dev/ttyWK1
K3	RS485_2_B1_CON_2_RS232_1_RX_R	
C4	RS485_1_A1_CON_1_RS232_2_TX_R	/dev/ttyWK3
D4	RS485_1_B1_CON_1_RS232_2_RX_R	

Pin definition

Pin number	Pin signal definition	Software device name
J3	RS485_2_A1_CON_2_RS232_1_TX_R	/dev/ttyWK1
K3	RS485_2_B1_CON_2_RS232_1_RX_R	
C4	RS485_1_A1_CON_1_RS232_2_TX_R	/dev/ttyWK3
D4	RS485_1_B1_CON_1_RS232_2_RX_R	

使用方法

步骤一： **机器端配置 (minicom 工具)**

通过 RS485 接口，将线束接在机器和电脑两端。确认连接正确，再进行配置。

minicom 工具配置方法：

打开终端输入：`sudo minicom -s`，（如果输入命令但显示找不到命令，则机器需要安装 minicom，安装命令为`sudo apt install minicom`）回车，按上下箭头，选择第三行 Serial port setup，回车，进入到端口设置界面如下：

Usage method

Step 1: Machine side configuration (minicom tool)

Connect the wire harness to both ends of the machine and computer through the RS485 interface. Ensure that the connection is correct and then configure the configuration.

minicom tool configuration method:

Open the terminal and type: `sudo minicom -s`, (If you enter the command but the command is not found, the machine needs to install minicom, the installation command is `sudo apt install minicom`) Enter, press the up and down arrow, select the third line Serial port setup, enter, The port Settings screen is displayed as follows:

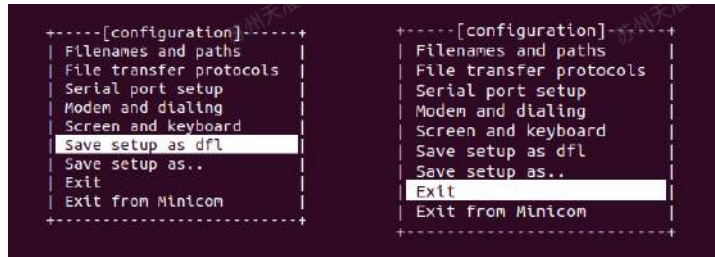


根据界面显示，按对应的英文字母改对应的数据，比如说想要改A行的数据则需要按a，修改完毕之后，按回车键返回上一步；修改E行波特率，则需要按e，修改完毕之后，按键盘AB键加減，改完按回车键返回上一步；修改Hardware Flow Control，按键盘 f 键切换Yes或者No。

修改完之后回车，再次回到以下界面，选择 Save setup as dfl 回车，再选择 Exit 回车。

According to the interface display, change the corresponding data according to the corresponding English letter, for example, if you want to change the data in line a, you need to press a, after modification, press Enter to return to the previous step; Change E line baud rate, you need to press e, after the modification is completed, press the keyboard AB key to add or subtract, after the change press Enter key to return to the previous step; Modify Hardware Flow Control and press f to switch to Yes or No.

After the modification, press Enter to return to the following screen again, select Save setup as dfl press enter, and then select Exit Press enter.



注意：RS232 对应的软件名： ttysWK1

RS485 对应的软件名： ttysWK3

设置波特率时，要求电脑端与机器端配置的**必须一致**，否则不能通信。

Hardware Flow Control 选项默认为 **Yes**，需改为 No，才可通信。

Note: RS232 corresponding software name: ttysWK1

RS485 software name: ttysWK3

When setting the baud rate, the configurations on the computer and the machine **must be consistent**; otherwise, communication cannot be performed.

The Hardware Flow Control option defaults to **Yes** and must be changed to No to enable communication.

步骤二：电脑端配置

1. windows系统

步骤一：下载串口调试工具，例如sscom。

步骤二：用 RS232/485 的线连在机器端和电脑端，在串口调试工具上配置相应的信息。

以sscom为例，配置界面如下：

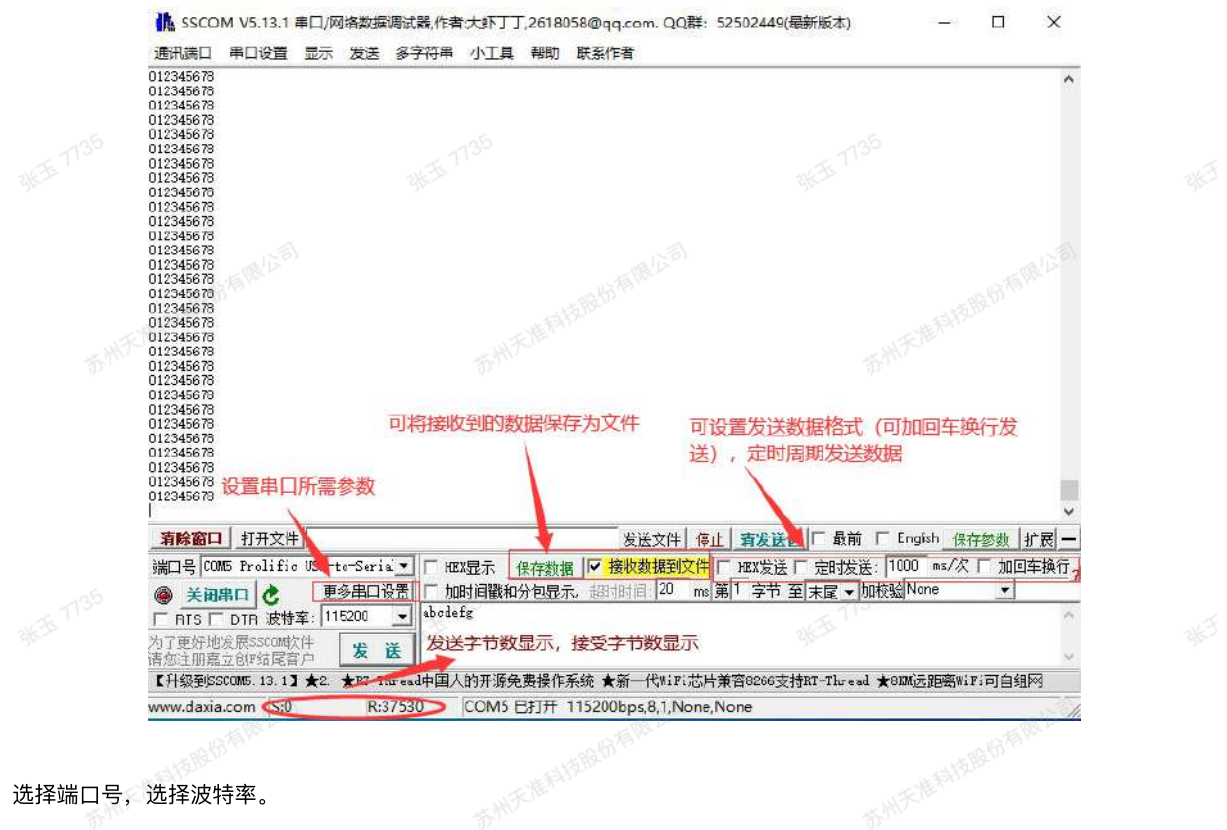
Step 2: Configure the PC

1. windows operating system

Step 1: Download the serial debugging tool, such as sscom.

Step 2: Connect the machine side and the computer side with RS232/485 line, and configure the corresponding information on the serial debugging tool.

Take sscom as an example. The configuration page is as follows:



选择端口号，选择波特率。

Select a port number and baud rate.

2. linux系统

步骤一：用 RS232/485 的线连在机器端和电脑端。

步骤二：配置参数（minicom工具）

(1) 打开终端输入：`sudo minicom -s`，回车，按上下箭头，选择第三行，回车，进入到端口设置界面如下：

2. linux system

Step 1: Connect the machine end and the computer end with RS232/485 wire.

Step 2: Configure parameters (minicom tool)

(1) Open the terminal and input: `sudo minicom -s`, press enter, press the up and down arrow, select the third line, press enter, enter to enter the port setting interface as follows:



(2) 重新打开一个终端输入: `watch -n 1 "lsusb"`, 插拔插串口接头, 确定插串口的名称

(2) Open a terminal again and input: `watch -n 1 "lsusb"`, plug and unplug the serial port connector, and determine the name of the serial port

(3) 在以下界面的A行修改为串口的实际名称。

(3) Change the actual name of the serial port in line A of the following page.

(4) 例如观察到串口的名称为 `ttyUSB0`, 则将A行修改为 `ttyUSB0`, 如下图:

(4) For example, if the name of the serial port is observed as `ttyUSB0`, change row A to `ttyUSB0`, as shown in the following figure:

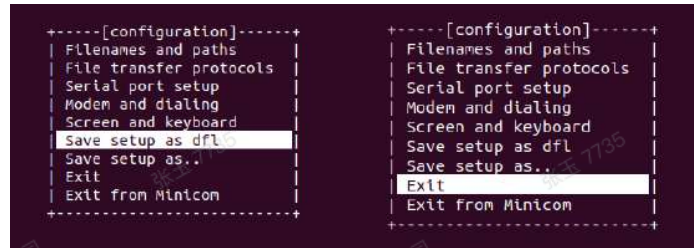


(5) 保存退出

修改完之后回车, 再次回到以下界面, 选择 `Save setup as dfl` 回车, 再选择 `Exit` 回车。

(5) Save and exit

After the modification, press Enter to return to the following screen again, select `Save setup as dfl` press enter, and then select `Exit` Press enter.



步骤三：收发检验

RS232 是全双工，可以同步收发，配置完成之后两边可以同时收发数据；

RS485 是半双工，不可以同步收发；

Step 3: Send and receive inspection

RS232 is full-duplex, which can be synchronous transceiver. After the configuration is complete, both sides can send and receive data at the same time.

RS485 is half-duplex and cannot be sent or received synchronously.

3. 串口同时使用

若在使用过程中需要同时使用两个串口，有两种使用方式：

方式一：每次开机执行指令：sudo jetson_clocks，开启最大性能模式

方式二：安装相关服务包：[tztek-jetson-service-cpufrequency-max-v1.0](#) 即可

3. Use the serial ports simultaneously

If you need to use two serial ports at the same time, there are two ways to use them:

Method 1: Run the sudo jetson_clocks commands each time you start the maximum performance mode

Method 2: Install the related service package: tztek-jetson-service-cpufrequency-max-v1.0

网络使用

Internet use

网口信息资源

GEACX1：带有4路M12千兆网络。用户可以根据需要自行选择网口进行使用。

独立千兆网（M12接口）x4

网络	个数	软件端口号	硬件名称	网卡信息
独立千兆	4	对外开放: eth0、eth1、eth2、eth3	LAN1 LAN2 LAN3 LAN4	原生网卡eth3+WX1860AL4 Gigabit Ethernet Controller
主板独立万兆	1	对外开放: xfi	10G	

Network port information resource

GEACX1: With a 4-way M12 Gigabit network. You can select network ports as required.

Independent Gigabit network (M12 interface) x4

network	number	Software port number	Hardware name	Network card information
Independent gigabit	4	Open to the public: eth0, eth1, eth2, eth3	LAN1 LAN2 LAN3 LAN4	Native NIC eth3+WX1860AL4 Gigabit Ethernet Controller
Mainboard independent 10 Gigabit	1	Open to the public: xfi	10G	

网络线束

Network harness

万兆以太网线束

10 Gigabit Ethernet harness

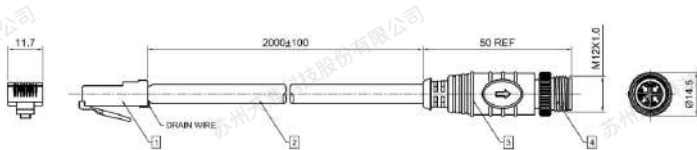
线束实物图

Diagram of wire harness



线束规格

Harness specification



M12-X端线序	1	2	3	4	5	6	7	8
RJ45端线序	1	2	3	6	7	8	5	4
线色	橙白	橙	绿白	绿	棕白	棕	蓝白	蓝

M12-X End cable sequence	1	2	3	4	5	6	7	8
RJ45 end cable sequence	1	2	3	6	7	8	5	4
Line color	orange and white	orange	green and white	green	Brown and white	brown	blue and white	blue

千兆以太网线束

线束实物图

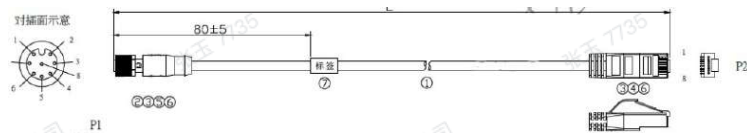
Gigabit Ethernet harness

Diagram of wire harness



线束规格

Harness specification



M12-A端线序	1	2	3	4	5	6	7	8
RJ45端线序	5	7	8	2	3	1	4	6
线色	白蓝	白棕	棕	橙	白绿	白橙	蓝	绿

M12-X End cablesequence	1	2	3	4	5	6	7	8
RJ45 endcable sequence	5	7	8	2	3	1	4	6
Line color	white and blue	white and brown	brown	orange	white and green	white and orange	blue	green

网络测试

默认出厂设置所有的对外开放接口为 dhcp 模式，可以通过路由器直接连接网口进行上网，使用命令 `ifconfig` 可以查看各网络情况。

使用路由器连接机器上的千兆网口会自动获取IP。

Network test

By default, all external interfaces are in dhcp mode. You can directly connect the network port to the router for Internet access. You can run the `ifconfig` command to check the network status.

Using a router to connect the gigabit network port on the machine automatically obtains the IP.

```
nvidia@nvidia-desktop:~$ ifconfig
dummy0: flags=195<UP,BROADCAST,RUNNING,NOARP> mtu 1500
        inet fe80::1578:16153:9ca2:11e4 prefixlen 64 scopeid 0x20<link>
        ether 0a:9d:6b:4f:14:78 txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 28 bytes 7572 (7.5 KB)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
        ether 48:b0:2d:3a:87:9b txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
        device interrupt 43

eth1: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
        ether 00:54:5a:18:00:21 txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
        device memory 0x123000000-123001ffff

eth2: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
        ether 00:54:5a:18:00:22 txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
        device memory 0x123002000-123003ffff

eth3: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
        ether 00:54:5a:18:00:23 txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
        device memory 0x123004000-123005ffff

eth4: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
        ether 00:54:5a:18:00:24 txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
        device memory 0x123006000-123007ffff
```

注意：测试有线网络的时候为避免其他网络影响要关掉无线网。（界面右上角的网络图标，点进去，打开网络，关掉wifi）

Note: When testing a wired network, turn off the wireless network to avoid other network effects. (The network icon in the upper right corner of the interface, tap it, turn on the network, turn off the wifi)

给机器接上网线(保证网线无损坏，可以正常上网)。

连接端（电脑端）RJ45接口显示灯亮表示正在连接，RJ45接口一般有两个灯左侧微绿色：正常亮起代表物理连通正常，右侧微黄色（或者橙色）：有数据流时会正常闪烁；如果不闪，一般是物理连接正常而对方的交换机、服务器无响应。

如果绿灯也不亮表示物理连接有问题，表示网络链路不通，需要排查网线连接。

Connect the network cable to the machine (ensure that the network cable is not damaged and can be connected to the Internet normally).

The RJ45 interface light on indicates that the connector is being connected. The RJ45 interface generally has two lights. The left side is green: the normal light indicates that the physical connection is normal. The right side is yellow (or orange) : When there is data flow, it blinks normally. If no, the physical connection is normal but the other switch or server does not respond.

If the green is not on, it indicates that the physical connection is faulty and the network link is unavailable. You need to check the network cable connection.

配置静态IP（使用 netplan）

netplan 是一个用于配置 linux 网络的简单工具。通过读取配置文件 `/etc/netplan/01-network-manager-all.yaml` 中对于各网络接口的配置，便可完成对各网络接口的配置。

Configuring a static IP Address (using netplan)

netplan is a simple tool for configuring linux networks. You can complete the configuration of each network interface by reading the configuration file `/etc/netplan/01-network-manager-all.yaml`.

操作步骤

修改配置文件

若修改 eth1 的 ip 地址为 192.168.1.197/24，打开配置文件后将 eth1 的 addresses 修改为 192.168.1.197/24，效果如下：

Modify the configuration file

If the IP address of eth1 is changed to 192.168.1.197/24, and the addresses of eth1 is changed to 192.168.1.197/24 after the configuration file is opened, the effect is as follows:

```
#打开配置文件（Open the configuration file）
sudo vim /etc/netplan/01-network-manager-all.yaml

#vim结果如下：（vim give the result as follows）

# Let NetworkManager manage all devices on this system

network:

version: 2
```

```
renderer: networkd
```

```
ethernets:
```

```
  eth1:
```

```
    addresses: [192.168.1.197/24]
```

```
    gateway4: 192.168.1.1
```

```
    nameservers:
```

```
      search: [lan]
```

```
      addresses: [114.114.114.114, 192.168.1.1]
```

```
  eth10:
```

```
    dhcp4: no
```

```
    dhcp6: no
```

```
    addresses: [192.168.44.100/24]
```

注意：使用时，请按照实际情况修改（改为自己想要配置的 ip）。如有疑问，可咨询 FAE。

Note: When using, please modify according to the actual situation (change to the IP you want to configure). If in doubt, consult the FAE

验证 IP(Authentication IP)

```
#测试网络配置 (Test network configuration)
```

```
sudo netplan try
```

```
#验证IP (Authentication IP)
```

```
ip a
```

```
#应用网络配置文件 (Application network configuration file)
```

```
sudo netplan apply
```

故障排查(troubleshooting)

Q1: 千兆以太网eth3、eth1出现联网不稳定的现象, 该如何解决?

A1: 可通过配置DNS解决千兆以太网 (eth3、eth1) 联网不稳定问题, DNS配置生效后千兆以太网eth1、eth3可稳定联网, 具体DNS配置方式如下:

复制代码

```
1.软件更新, 若未安装resolvconf, 首先安装resolvconf
sudo apt update
sudo apt install resolvconf

2.检查解析服务的运行状态, 确保resolvconf.service处于启动状态 (active)
sudo systemctl status resolvconf.service

#若resolvconf.service处于非工作 (dead) 状态, 执行如下任意一条指令启动该解析服务
sudo systemctl start resolvconf.service
sudo systemctl enable resolvconf.service

3.在resolv.conf.d/head 配置文件中添加本设备的DNS地址,
sudo vim /etc/resolvconf/resolv.conf.d/head

nameserver 114.114.114.114
nameserver 8.8.8.8      #添加的本设备的DNS地址根据用户的实际情况决定

4.执行指令强制 resolvconf运行更新脚本, 更新resolvconf
sudo resolvconf --enable-updates
sudo resolvconf -u

5.修改 /etc/resolv.conf 文件链接
ln -sf /run/resolvconf/resolv.conf /etc/resolv.conf

6.重启设备, 查看/etc/resolv.conf中是否新增2行添加的DNS配置信息, 如下图
(nameserver 114.114.114.114、nameserver 8.8.8.8)
cat /etc/resolv.conf
```

Q1: Gigabit Ethernet eth3, eth1 network instability phenomenon, how to solve?

A1: You can configure DNS to resolve the unstable networking of Gigabit Ethernet (eth3 and eth1). After the DNS configuration takes effect, Gigabit Ethernet eth1 and eth3 can be stably connected. The DNS configuration is as follows:

复制代码

```
If resolvconf is not installed, install resolvconf first          sudo apt update
sudo apt install resolvconf
Check the running status of the resolvconf.service and ensure that the resolvconf.service is
active.                  sudo systemctl status resolvconf.service

                        #If resolvconf.service is in the dead state, run one of the
following commands to start the resolvconf.service
sudo systemctl start resolvconf.service          sudo systemctl enable
resolvconf.service
Add the DNS address of the device to the resolv.conf.d/head configuration file sudo vim
/etc/resolvconf/resolv.conf.d/head                  nameserver 114.114.114.114
nameserver 8.8.8.8      #The DNS address of the device to be added
depends on the user's actual situation
Run the command to force resolvconf to run the update script to update resolvconf
sudo resolvconf --enable-updates
```

```

sudo resolvconf -u
ln -sf /run/resolvconf/resolv.conf /etc/resolv.conf
Restart the device and check whether the added DNS configuration information in two lines is added to /etc/resolv.conf, as shown in the following figure (nameserver 114.114.114.114, nameserver 8.8.8.8)
cat /etc/resolv.conf

```

```

nvidia@nvidia-desktop:~$ sudo vim /etc/resolvconf/resolv.conf.d/head
nvidia@nvidia-desktop:~$ sudo ln -sf /run/resolvconf/resolv.conf /etc/resolv.conf
nvidia@nvidia-desktop:~$ cat /etc/resolv.conf
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
# DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
# 127.0.0.53 is the systemd-resolved stub resolver.
# run "systemd-resolve --status" to see details about the actual nameservers.

nameserver 114.114.114.114
nameserver 8.8.8.8
nameserver 127.0.0.53
nvidia@nvidia-desktop:~$
nvidia@nvidia-desktop:~$

```

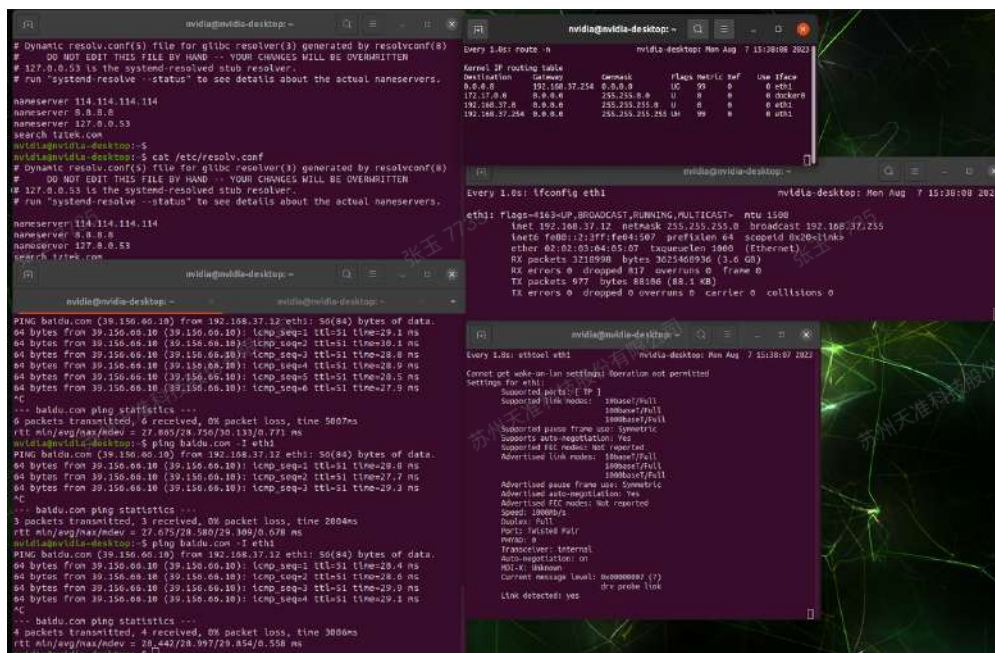
```

nvidia@nvidia-desktop:~$ route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 192.168.37.254 0.0.0.0 UG 103 0 0 eth3
192.168.37.0 0.0.0.0 255.255.0.0 U 0 0 0 dockers
192.168.37.0 0.0.0.0 255.255.255.0 U 103 0 0 eth3
nvidia@nvidia-desktop:~$

nvidia@nvidia-desktop:~$ ifconfig eth3
eth3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.37.92 netmask 255.255.0.0 broadcast 192.168.37.255
link/ether 08:00:2d:ae:2d:7e db:fa:8a:30:ae:0d:00:00:00:00:00:00:00:00
ether 08:00:2d:ae:2d:7e db:fa:8a:30:ae:0d:00:00:00:00:00:00:00:00
RX packets 2300200 bytes 260790225 (2.4 GB)
RX errors 0 dropped 195 overruns 0 frame 0
TX packets 369 bytes 14942 (14.5 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

nvidia@nvidia-desktop:~$ ethtool eth3
Every 1.0s: ethtool eth3 nvidia-desktop: Mon Aug 7 16:11:07 2023
Cannot get wake-on-lan settings: Operation not permitted
Settings for eth3:
Supported ports: [ TP MII FIBRE ]
Supported link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Half 1000baseT/Full
Supported pause frame use: Symmetric Receive-only
Supports auto-negotiation: Yes
Supported FEC modes: Not reported
Advertised link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Half 1000baseT/Full
Advertised pause frame use: No
Advertised auto-negotiation: Yes
Advertised FEC modes: Not reported
Link partner advertised link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Half
Link partner advertised pause frame use: No
Link partner advertised auto-negotiation: Yes
Link partner advertised FEC modes: Not reported
Speed: 1000Mb/s
Duplex: Full
Port: Twisted Pair
Patch: 0
Transceiver: Internal
Auto-negotiation: on
MDIX: Unknown
Current message level: 0x00000000 (0)
Link detected: yes

```

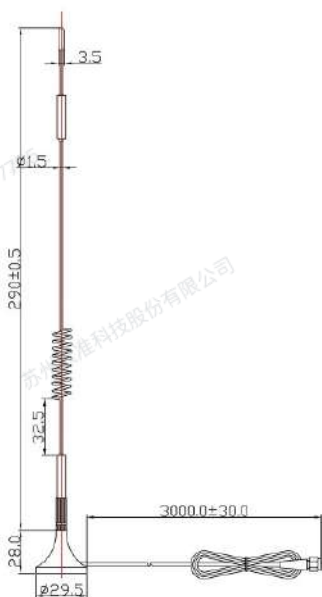


4G

本产品支持4G网络（选配），支持中国移动、中国联通、中国电信三大运营商的4G网络服务。（使用4G网络前，请插入4G卡，并且正确接好天线。）

This product supports 4G network (optional) and 4G network services from China Mobile, China Unicom, and China Telecom. (Before using 4G network, please insert the 4G card and connect the antenna correctly.)

天线图 (Antenna diagram)



模块确认 (Module confirmation)

可以通过 `lsusb` 命令检查设备模块信息验证驱动是否加载正常，目前使用的4G模块为OMEGA（龙尚）模块。如下图可以看到这台机器的4G模块为OMEGA（龙尚）模块。

You can run the `lsusb` command to check the device module information to verify that the driver is loaded properly. The current 4G module is the OMEGA (Longshang) module. As can be seen in the following figure, the 4G module of this machine is OMEGA (Longshang) module.

```
nvidia@nvidia-desktop: ~
nvidia@nvidia-desktop:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 1c9e:9b3c OMEGA TECHNOLOGY
Bus 001 Device 002: ID 25a7:fa23
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
nvidia@nvidia-desktop:~$
```

使用步骤 (Procedure of use)

1. 下载并安装最新版的4G服务包

经过测试，系统自带的移动网络ppp拨号服务不稳定，不建议客户生产使用。为此，天准开发了拨号程序，经过6000次测试，可100%自动拨号上网，建议使用。

1. Download and install the latest 4G service pack

After testing, the mobile network ppp dial-up service of the system is unstable, and it is not recommended for customers to produce and use. To this end, Tianjun developed a dial-up program, after 6,000 tests, 100% automatic dial-up

Internet access, recommended use.

(1) 下载并安装最新版的4G服务包。(Download and install the latest 4G service pack.)

服务包下载 (Service pack download) :

- [tztek-jetson-service-net-mobile-connect-v1.6.1.deb](#)

(2) 打开终端，执行以下命令进行安装 (Open the terminal and run the following command to install) :

```
sudo dpkg -i tztek-jetson-service-net-mobile-connect-v1.6.1.deb
```

备注：出厂默认会安装最新版本服务包。

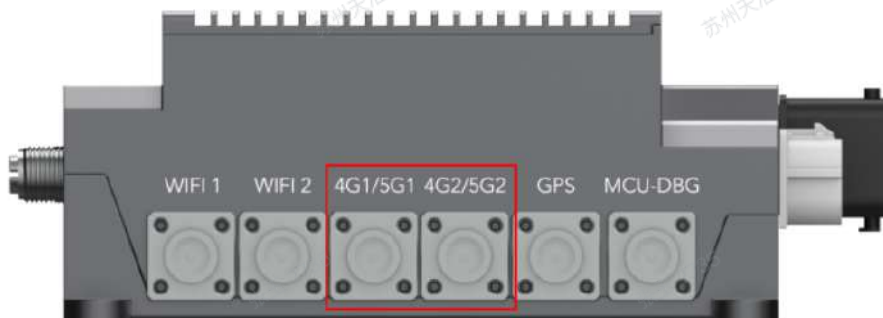
Note: The latest version of the service pack is installed by default.

2. 安装4G天线

本机器使用前需要插入天线，4G两根天线安装位置如下图红色箭头所指，具体接口说明可参考[接口说明-背面接口](#)。

2. Install the 4G antenna

The antenna needs to be inserted before the use of this machine. The installation position of the two 4G antennas is indicated by the red arrow in the figure below. For the specific interface description, refer to the [interface description - Rear interface](#).



3. 插入SIM卡

先准备一张SIM卡，保证sim卡的大小恰好适应卡槽的尺寸，过小的sim卡无法正确卡入卡槽或者按进去之后难以取出，需要返厂维修。

3. Insert the SIM card

Prepare a SIM card first to ensure that the size of the sim card is just suitable for the size of the card slot. If the sim card is too small, it cannot be correctly inserted into the card slot or it is difficult to take out after being pressed in, and it needs to be returned to the factory for maintenance.



带有芯片的一面朝上，有缺口的一侧朝右上角，如下图：

The side with the chip faces upward, and the side with the notch faces the upper right corner, as shown in the following figure:



将卡插入，用力按进去，会发出“啪”的一声，固定在卡槽中（如果插卡比较费力，可以借助牙签、笔头之类的工具辅助）。需要取出SIM卡时，往里一推，卡片自动弹出，拿出即可。

Insert the card, press it hard, it will make a "snap" sound, and it will be fixed in the card slot (if it is difficult to insert the card, you can use tools such as toothpicks and pen tips to assist). When you need to take out the SIM card, push it in and the card pops out automatically.

SIM卡正确卡入卡槽效果如图所示：

The effect of correctly inserting the SIM card into the card slot is shown as follows:



4. 断电重启

4G模块自动拨号服务需要重新启动生效，重启成功后，开机自启拨号服务。

4. Power off and restart

The automatic dial service of the 4G module needs to be restarted to take effect. After the restart is successful, the dial service is automatically started after the device is powered on.

验证服务 (Verification service)

查看4G服务运行状态 (View the 4G service running status)

#输入命令 (Input command)

```
sudo systemctl status tztek-jetson-service-net-mobile-connect.service
```

如下图所示，可在命令返回结果中获取：服务状态、SIM卡状态、网口信息、获取的IP以及信号强度等信息。

Service status, SIM card status, network port information, obtained IP address, and signal strength can be obtained in the command output, as shown in the following figure.

```
nvidia@nvidia-desktop:~$ sudo systemctl status tztek-jetson-service-net-mobile-connect.service
[sudo] password for nvidia:
● tztek-jetson-service-net-mobile-connect.service - tztek-jetson-service-net-mobile-connect for tztek platform
   Loaded: loaded (/lib/systemd/system/tztek-jetson-service-net-mobile-connect.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2021-07-04 23:56:22 EDT; 1min 30s ago
     Main PID: 5475 (tztek-jetson-se)
       Tasks: 2 (limit: 4915)
      CGroup: /system.slice/tztek-jetson-service-net-mobile-connect.service
              └─5475 /bin/bash /usr/bin/tztek-jetson-service-net-mobile-connect
                  └─8835 ping -c 4 -I eth5 114.114.114.114

Jul 04 23:56:52 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: at-port is ttyUSB1 ...
Jul 04 23:56:53 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: AT+CPIN?
Jul 04 23:56:53 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: +CPIN: READY
Jul 04 23:56:53 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: [18 blob data]
Jul 04 23:56:53 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: OK
Jul 04 23:56:53 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: generate eths ...
Jul 04 23:57:01 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: get ip: 10.35.193.103 ...
Jul 04 23:57:01 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: Dial-up Successfully
Jul 04 23:57:37 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: 2021-07-04 23:57:37 | Network Connecting
Jul 04 23:57:50 nvidia-desktop tztek-jetson-service-net-mobile-connect[5475]: 2021-07-04 23:57:50 | Network Connecting
```

查询4G网络对应网口 (Query the network port corresponding to the 4G network)

#使用如下命令查看4G模块对应的网卡接口 (Run the following command to view the network adapter port corresponding to the 4G module) :

```
ls /sys/bus/usb/devices/1-4/1-4:1.4/net
```

故障排查 (troubleshooting)

Q1: 查询到4G服务运行状态为active (running) 但是获取不到ip, 该如何解决?

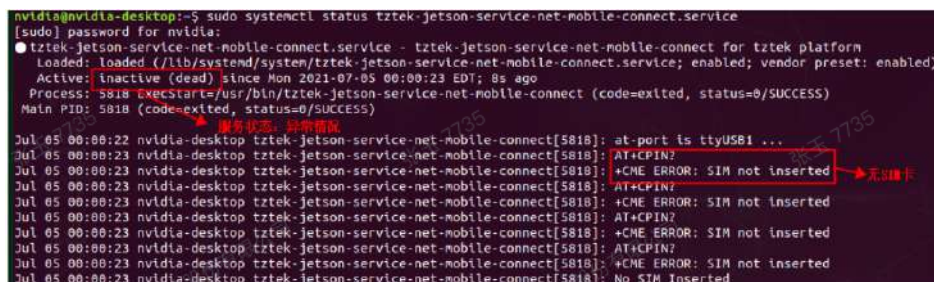
A1: 信号差影响, 检查是否连接天线。SIM卡欠费, 检查卡内流量是否可用。

Q1: The 4G service status is active (running) but the ip address cannot be obtained. How to solve this problem?

A1: Signal difference, check whether the antenna is connected. If the SIM card is not charged, check whether the data in the card is available.

Q2: 服务状态为inactive (dead) , 如下图所示, 该如何解决?

Q2: The service state is inactive (dead), as shown in the following figure. How can I solve the problem?



```
nvidia@nvidia-desktop:~$ sudo systemctl status tztek-jetson-service-net-mobile-connect.service
[sudo] password for nvidia:
● tztek-jetson-service-net-mobile-connect.service - tztek-jetson-service-net-mobile-connect for tztek platform
   Loaded: loaded (/lib/systemd/system/tztek-jetson-service-net-mobile-connect.service; enabled; vendor preset: enabled)
   Active: inactive (dead) since Mon 2021-07-05 00:00:23 EDT; 8s ago
     Process: 5818 ExecStart=/usr/bin/tztek-jetson-service-net-mobile-connect (code=exited, status=0/SUCCESS)
    Main PID: 5818 (code=exited, status=0/SUCCESS)
      Service Status: Error status
Jul 05 00:00:22 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: at-port is ttyUSB1 ...
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: AT+CPIN?
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: +CME ERROR: SIM not inserted
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: AT+CPIN?
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: +CME ERROR: SIM not inserted
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: AT+CPIN?
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: +CME ERROR: SIM not inserted
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: AT+CPIN?
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: +CME ERROR: SIM not inserted
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: AT+CPIN?
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: +CME ERROR: SIM not inserted
Jul 05 00:00:23 nvidia-desktop tztek-jetson-service-net-mobile-connect[5818]: No SIM Inserted
```

A2: 可以通过查看4G服务状态获取更多信息, 在服务状态信息中得知, "SIM not inserted", 此种情况为未识别SIM卡, 因此首先检查SIM卡是否已经插入, 如果已插入需要再次检查是否插入正确, 可参考使用说明中的"正确插入SIM卡" 章节。

A2: You can obtain more information by looking at the status of the 4G service. In this case, the SIM card is not identified, so first check whether the SIM card is inserted. If it is inserted, you need to check again whether the SIM card is inserted correctly.

Q3: 多次断电重启以后突然出现无法拨号上网的情况, 怎么办?

A3: 可通过查看4G服务运行状态, 在服务状态正常且SIM插入正确的情况下, 将机器挪到4G网络信号较好的地方, 手动拨号上网。同时可使用手机检测当地4G网络信号强度来进一步确认是否为网络信号强度影响。

Q3: What should I do if the dial-up Internet connection fails suddenly after several power outages and restarts?

A3: You can check the running status of 4G service, and when the service status is normal and the SIM is inserted correctly, move the machine to a place with good 4G network signal and dial up the Internet manually. At the same time, mobile phones can be used to detect the signal strength of the local 4G network to further confirm whether the network signal strength is affected.

Q4: 如何测试GEACX1盒子的4G网络速度?

A4: 【网络】 网络相关—网络速度测试, 可用网络测速网址 www.speedtest.cn 进行测试。

Q4: How to test the 4G network speed of the GEACX1 box?

A4: 【 Network 】 Network related - Network speedtest, network speedtest website www.speedtest.cn.

Q5: 如何得知4G模块当前是否运行正常?

A5: 通过系统日志查看命令来查看4G服务模块在运行中的日志，根据日志内容获取更多模块运行信息。

Q5: How do I know if the 4G module is working properly?

A5: Run the system log view command to view the running logs of the 4G service module, and obtain more module running information according to the log content.

#输入命令 (# Enter command)

```
journalctl -u tztek-jetson-service-net-mobile-connect.service
```

```
-- Logs begin at Sun 2021-09-05 16:40:11 CST, end at Sun 2021-09-05 17:12:02 CST. --
9月 05 16:40:18 nvidia-desktop systemd[1]: Started tztek-jetson-service-net-mobile-connect for tztek platform.
9月 05 16:40:29 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: at+port is /dev/ttyUSB1 ...
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: at+cfun=0
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: ^MODE: 0
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: OK
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: at+cfun=1
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: OK
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: ^MODE: 9
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: AT+CPIN?
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: +CPIN: READY
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: OK
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: AT+PSRAT
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: +PSRAT: LTE FDD
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: OK
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: AT+CSQ
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: +CSQ: 31,49
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: OK
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: AT+SQCRMCALL=1,1
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: $QCRMCALL: 1, V4
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: [iB blob data]
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: OK
9月 05 16:40:45 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: generate eth2 ...
9月 05 16:40:57 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: get Ip: 10.125.38.252 ...
9月 05 16:40:57 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: Dial-up Successfully
9月 05 16:41:17 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:41:17 Network Connecting
9月 05 16:41:34 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:41:34 Network Connecting
9月 05 16:41:51 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:41:51 Network Connecting
9月 05 16:42:08 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:42:08 Network Connecting
9月 05 16:42:25 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:42:25 Network Connecting
9月 05 16:42:42 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:42:42 Network Connecting
9月 05 16:42:59 nvidia-desktop tztek-jetson-service-net-mobile-connect[6270]: 2021-09-05 16:42:59 Network Connecting
```

WIFI

GEACX1设备中的 wifi 5 模块包括两种工作模式ap和sta，ap 即无线接入点，是一个无线网络的创建者，是网络的中心节点。一般家庭或办公室使用的无线路由器就是一个ap；sta 指每一个连接到无线网络中的终端（如笔记本电脑、PDA及其它可以联网的用户设备）都可称为一个站点。

The wifi 5 module in the GEACX1 device includes two working modes ap and sta, ap is the wireless access point, which is the creator of a wireless network and the central node of the network. A typical wireless router used in a home or office is an ap; sta means that every terminal connected to a wireless network (such as laptops, PDAs, and other Internet-connected user devices) can be called a site.

使用步骤

Procedure of use

WIFI天线实物图 (WIFI antenna physical picture)



下图为未安装WiFi天线时的信号强度 (The following figure shows the signal strength when no WiFi antenna is installed)



下图为安装了WiFi天线时的信号强度 (The following figure shows the signal strength when the WiFi antenna is installed)



使用方法

信号查看 (Signal viewing)

输入如下代码 (Enter the following code)

#信号查看 (Signal viewing)

iwconfig

#动态查看wifi信号 (View wifi signals dynamically)

watch -n 1 "iwconfig"

sta模式设定

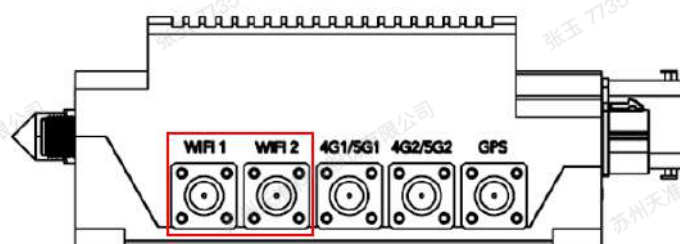
使用图形界面的NetworkManager管理使用wifi功能

sta mode setting

NetworkManager uses a graphical interface to manage the use of wifi capabilities

步骤一：接入wifi天线

Step 1: Access the wifi antenna

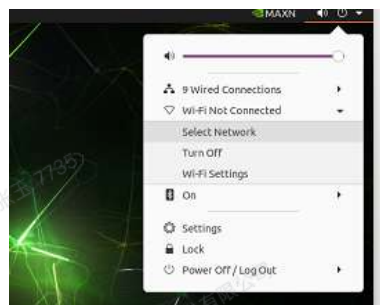


步骤二：连接无线网

点击开机界面右上角的网络连接图标，寻找待连接的WIFI网络单击，并在弹出的密码提示框中输入WIFI密码，点击Connect按钮即可连接，如下图所示：

Step 2: Connect to the wireless network

Click the network connection icon in the upper right corner of the startup interface, find the WIFI network to be connected, and click, enter the WIFI password in the pop-up password prompt box, and click the Connect button to connect, as shown in the following picture:





步骤三：验证是否可以上网

- (1) 打开终端，输入：`ifconfig`，检查 `mlan0` 这边获取到了ip。
- (2) 在终端中输入：`ping www.baidu.com`，如下图所示则为机器已经正常连接网络。

Step 3: Verify whether you can access the Internet

- (1) Open the terminal, enter: `ifconfig`, and check that the IP is obtained from `mlan0`.
- (2) Enter the `ping www.baidu.com` in the terminal. As shown in the following figure, the machine is connected to the network normally.

```
nvidia@nvidia-desktop: ~  
nvidia@nvidia-desktop:~$ ping baidu.com  
PING baidu.com (220.181.38.148) 56(84) bytes of data:  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=1 ttl=47 time=84.8 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=2 ttl=47 time=84.4 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=3 ttl=47 time=84.5 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=4 ttl=47 time=83.6 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=5 ttl=47 time=93.3 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=6 ttl=47 time=91.7 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=7 ttl=47 time=94.0 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=8 ttl=47 time=91.5 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=9 ttl=47 time=89.0 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=10 ttl=47 time=86.3 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=11 ttl=47 time=86.5 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=12 ttl=47 time=84.8 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=13 ttl=47 time=65.3 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=14 ttl=47 time=63.7 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=15 ttl=47 time=82.5 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=16 ttl=47 time=58.4 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=17 ttl=47 time=59.9 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=18 ttl=47 time=58.7 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=19 ttl=47 time=57.6 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=20 ttl=47 time=55.1 ms  
64 bytes from 220.181.38.148 (220.181.38.148): icmp_seq=21 ttl=47 time=65.3 ms
```

故障排查 (Troubleshooting)

若 wifi 无法上网，需故障排查

If the wifi cannot be accessed, rectify the fault

Q1: 信号不好

A1: 请查看是否连接天线，或者wifi信号质量是否较强。

Q1: The signal is bad

A1: Please check whether the antenna is connected, or whether the wifi signal quality is strong.

Q2: 网络优先级错误

A2: 使用命令 `route -n` 查看路由表，看可连接外网的IP地址的Metric值是否为最小。

Q2: The network priority is incorrect

A2: Run the `route -n` command to check whether the Metric value of the IP address that can connect to the Internet is the smallest.

FAN

风扇控制及使用方法

Fan control and usage methods

服务包下载 (Service pack download) : [tztek-jetson-service-fan-v2.1.deb](#)

控制风扇转动 (Control fan rotation) :

```
# 安装风扇服务包 (Install the fan service package)

sudo dpkg -i tztek-jetson-service-fan-v2.1.deb

# 修改温度控制逻辑 (Example Modify the temperature control logic)

sudo vim /etc/tztek/tztek-jetson-service-fan/config.json

-----

{

"FAN_OFF_TEMP":20,

"FAN_MAX_TEMP":50,

"UPDATE_INTERVAL":2,

"MAX_PERF":1
```

```
}
```

```
# 重启风扇服务 (Restart fan services)
```

```
sudo systemctl restart tztek-jetson-service-fan.service
```

FAN_OFF_TEMP : CPU 温度低于该温度(°C) 风扇停止转动。**FAN_MAX_TEMP**: CPU 温度高于该温度(°C) 风扇开始满功率转动。

FAN_OFF_TEMP: The fan stops running when the CPU temperature is lower than the temperature (°C).

FAN_MAX_TEMP: When the CPU temperature is higher than the threshold (°C), the fan starts to work with full power.

UPDATE_INTERVAL : 告诉服务多久更新一次风扇温度 (单位是秒) 。

UPDATE_INTERVAL: tells the service how often to update the fan temperature (unit: second).

把**FAN_MAX_TEMP** 和**FAN_OFF_TEMP**修改成自己想要的转动和停止温度即可, 保存后restart 重启服务

重启风扇服务后就可以正常根据温度转动 (您可以根据自己的需要修改温度数值, 可以支持整数和浮点数, 温度传感器的精度0.5摄氏度) :

(ps: 如果需要风扇一直转动可以把**FAN_MAX_TEMP**的数值写到**30°C**, 保证机器上电时 CPU 温度就达到了转动温度即可)

Set **FAN_MAX_TEMP** and **FAN_OFF_TEMP** to the desired rotation and stop temperatures, save the Settings, and restart the service

After restarting the fan service, the fan can rotate normally according to the temperature (you can change the temperature value according to your needs, which can be integer or floating point number, and the accuracy of the temperature sensor is 0.5 degrees Celsius) :

(ps: If the fan needs to keep running, the value of **FAN_MAX_TEMP** can be written to **30 ° C** to ensure that the CPU temperature reaches the rotation temperature when the machine is powered on.)

MCU

本次小卡出厂主要集成RTC显示, can数据收发, gpio状态读取, lin环收发, 核心板与MCU通信五大功能, 客户可根据需求自行开发其它功能, 以下为场内测试结果。

This small card factory mainly integrates RTC display, can data transceiver, gpio status reading, lin ring transceiver, core board and MCU communication five functions, customers can develop other functions according to needs, the following is the test results.

RTC显示 (RTC display)

机器上电串口打印时间信息为显示正常

The time displayed by the serial port on the machine is normal

gpio状态读取 (gpio status read)

将四路gpio输入输出端对连，串口打印输入端读取输出端引脚状态值呈高低电平变化为正常

Connect the four gpio input and output terminals, and read the pin status value of the output terminal at the serial port print input terminal as a high or low level change to normal

```
gpio_dev_0 read value = 1
gpio_dev_1 read value = 1
lin1 write up success
lin1 tx data success
lin1 rx data success
gpio_dev_2 read value = 1
LIN2 get_status:
-----lin2 rx data:0x0f 0x7f 0x0f 0x0f 0x0f 0x0f LIN2 receive over-----lin2 re
x data:0x0f 0x7f 0x0f 0x0f 0x0f 0x0f LIN2 receive over-----LIN2 get_status:
lin1 rx data:0x0f 0x7f 0x0f 0x0f 0x0f 0x0f
gpio_dev_3 read value = 1
lin_recv_frame_test_lin2_start
lin2 write up success
gpio_dev_4 read value = 0
gpio_dev_5 read value = 0
gpio_dev_6 read value = 0
gpio_dev_7 read value = 0
```

lin环测试

将lin1与lin2对连，串口打印如下log信息为显示正常

lin ring test

Connect lin1 to lin2. The following log information is displayed on the serial port

```
lin1 write up success
lin1 tx data success
LIN2 get_status:
-----lin2 rx data:0x0f 0x7f 0x0f 0x0f 0x0f 0x0f LIN2 receive over-----
LIN2 get_status:
lin1 rx data:0x0f 0x7f 0x0f 0x0f 0x0f 0x0f
lin_recv_frame_test_lin2_start
lin2 write up success
rtc_test
2022-12-31 23:58:46.152976
lin1 write up success
lin1 tx data success
LIN2 get_status:
-----lin2 rx data:0x0f 0x7f 0x0f 0x0f 0x0f 0x0f LIN2 receive over-----
```

通信测试

设置核心板静态IP为192.168.55.xxx, ping小卡IP地址192.168.55.110, 能ping通为正常

Communication test

Set the static IP address of the CPU module to 192.168.55.xxx and ping the IP address of the CPU module to 192.168.55.110. If the IP address can be pinged, the operation is normal

网卡半双工协议适配 (Nic half-duplex protocol adaptation)

下载地址 (Download address) : tstek-jetson-service-wx1860-half-link-1.0.deb

在使用过程中, 若需适配半双工协议外设, 可根据以上地址下载相关deb包, 安装配置完成后重启服务进行使用。WX1860小卡默认支持全双工协议。

If you need to adapt the half-duplex protocol peripheral, download the deb package based on the preceding address, and restart the service after the installation and configuration is complete. The WX1860 card supports the full-duplex protocol by default.

```
# 进入配置路径 (Enter configuration path)

cd /opt/tstek/tstek-jetson-service-wx1860-half-link

# 修改网口配置 (Modifying Network Port Configurations)

vim config
```

示例: 若外设速率为100, 协议为半双工, 使用硬件网口为LAN1 (对应软件接口为eth0), 可按以下方法对网口进行配置, 配置完成后重启服务即可。

Example: If the peripheral rate is 100, the protocol is half-duplex, and the used hardware network port is LAN1 (the corresponding software interface is eth0), the network port can be configured according to the following method. After the configuration is completed, restart the service.



同理，若使用硬件网口为LAN2，则把对应软件接口 由eth0改为eth1，以此类推。

若在使用过程中外设切换为全双工协议模式，有两种处理方式：

方式一：卸载deb包并重启机器；

方式二：把配置文件中的off改为on，网口速率配置与外设保持一致（若外设速率未知，则删除对应网口配置信息），配置完成后重启服务。

Similarly, if the hardware network port is LAN2, change the corresponding software interface from eth0 to eth1, and so on.

If you switch to the full-duplex protocol mode during use, there are two processing methods:

Method 1: Uninstall the deb package and restart the machine.

Method 2: Change the off value in the configuration file to on, and keep the network port rate consistent with that of the peripheral. (If the rate of the peripheral is unknown, delete the corresponding network port configuration.) After the configuration is complete, restart the service.

EMMC使用寿命分析（EMMC Service Life Analysis）

下载地址（Download address）：[tztek-jetson-tool-eMMC-v1.3.deb](https://www.tztek.com.cn/download/tztek-jetson-tool-eMMC-v1.3.deb)

工具使用方法：

安装：sudo dpkg -i tztek-jetson-tool-eMMC-v1.3.deb

运行：

sudo tztek-jetson-tool-eMMC-life-test

The mmc0 ext_csd is OK

The mmc0 pre_EoL_info is OK

SLC 0%~10% of EMMC device life time used.

MLC 10%~20% of EMMC device life time used.

表示：SLC使用寿命为0%~10%，MLC使用寿命为10%~20%。

How to use tools:

Install: `sudo dpkg -i tztek-jetson-tool-eMMC-v1.3.deb`

Run:

`sudo tztek-jetson-tool-eMMC-life-test`

The mmc0 ext_csd is OK

The mmc0 pre_Eol_info is OK

SLC 0%~10% of EMMC device life time used.

MLC 10%~20% of EMMC device life time used.

The service life of SLC is 0% to 10%, and the service life of MLC is 10% to 20%.

```
nvidia@nvidia-desktop:~/yuhaibao$ sudo tztek-jetson-tool-eMMC-life-test
The mmc0 ext_csd is OK
The mmc0 pre_Eol_info is OK
SLC 0%~10% of EMMC device life time used.
MLC 10%~20% of EMMC device life time used.
nvidia@nvidia-desktop:~/yuhaibao$
```